



POLITECHNIKA
LUBELSKA
WYDZIAŁ MATEMATYKI
I INFORMATYKI TECHNICZNEJ

Katedra Matematyki Stosowanej

Praca inżynierska

na kierunku *inżynieria i analiza danych*

Analiza skuteczności wybranych metod osadzania słów w zadaniach przetwarzania języka naturalnego

Analysis of the Effectiveness of Selected Word Embedding Methods in Natural Language Processing Tasks

Mateusz Drozd

numer albumu: 100966

promotor: dr inż. Magdalena Jastrzębska

promotor pomocniczy: mgr inż. Michał Błaszczkowski

Lublin 2026

Spis treści

Wstęp	5
Rozdział 1. Podstawy przetwarzania języka naturalnego	7
1.1. Definicja NLP oraz jego znaczenie w obszarze sztucznej inteligencji	7
1.2. Ewolucja przetwarzania języka naturalnego	8
1.3. Rola sztucznej inteligencji w zadaniach przetwarzania języka naturalnego	11
Rozdział 2. Metody reprezentacji tekstu w NLP	15
2.1. Klasyczne metody reprezentacji tekstu	15
2.2. Statyczne metody reprezentacji tekstu	18
2.3. Kontekstowe metody reprezentacji tekstu	22
Rozdział 3. Wybrane architektury i algorytmy przetwarzania języka naturalnego	29
3.1. Tłumaczenie maszynowe	29
3.2. Algorytmy uczenia maszynowego w klasyfikacji tekstu	34
Rozdział 4. Analiza wyników i implementacja aplikacji	39
4.1. Zadanie klasyfikacji tekstu	39
4.1.1. Metryki ewaluacyjne modeli	41
4.1.2. Podejścia statystyczne	45
4.1.3. Podejścia osadzeń statycznych	47
4.1.4. Podejścia osadzeń kontekstowych	50
4.2. Weryfikacja jakości modeli tłumaczeniowych	59
4.3. Implementacja modułu rozpoznawania emocji i tłumaczenia tekstu	62
Podsumowanie i wnioski	69
Bibliografia	71
Spis rysunków	75
Spis tabel	77
Załączniki	79

Streszczenie (Summary)	81
---	----

Wstęp

Obecnie zrozumienie sposobu w jaki maszyny interpretują tekst oraz umiejętność tworzenia systemów zdolnych do rozpoznawania emocji i tłumaczenia międzyjęzykowego stanowią fundament nowoczesnych systemów informatycznych. Ewolucja od klasycznego programowania do zaawansowanych metod uczenia maszynowego otworzyła nowe możliwości analizy danych tekstowych, umożliwiając opracowywanie rozwiązań cechujących się coraz wyższą precyzją.

Praca poświęcona jest analizie i porównaniu skuteczności wybranych metod osadzania słów w zadaniu klasyfikacji emocji, a także weryfikacji jakości współczesnych modeli tłumaczenia maszynowego. W części teoretycznej przedstawiono szczegółowy opis ewolucji przetwarzania języka naturalnego. Kluczowym elementem tej części jest omówienie metod reprezentacji tekstu, począwszy od rozwiązań statystycznych, poprzez modele statyczne, aż po zaawansowane metody kontekstowe. Szczególną uwagę poświęcono architekturze Transformer oraz opartym na niej modelom, takim jak: BERT, RoBERTa czy GPT. Uzupełnieniem części teoretycznej jest opis ewolucji tłumaczenia maszynowego, od sieci rekurencyjnych po nowoczesne modele, takie jak mBART oraz M2M-100. Ponadto zaprezentowano charakterystykę wybranych algorytmów służących do klasyfikacji tekstu, w tym regresji logistycznej, algorytmu XGBoost oraz sieci neuronowej MLP.

W części praktycznej wykorzystano wyżej wymienione algorytmy uczenia maszynowego do klasyfikacji emocji w tekście. Celem badań było porównanie skuteczności różnych metod reprezentacji tekstu. Istotnym elementem badań była także weryfikacja jakości etykietowania danych, obejmująca identyfikację błędnych oznaczeń oraz ich automatyczną korektę przy użyciu autorskiego systemu opartego na modelu GPT-4o-mini. Ostatecznie modele wytrenowano na zbiorze oryginalnym oraz na zbiorze z poprawionymi etykietami, a następnie z wykorzystaniem metryk F1-macro, Accuracy oraz Balanced Accuracy porównano otrzymane wyniki. Ponadto dokonano oceny jakości wybranych modeli tłumaczeniowych, uwzględniając zarówno dokładność przekładu, jak i zachowanie semantyki tekstu.

Integralną częścią pracy jest opis implementacji autorskiej aplikacji w środowisku Streamlit, która łączy funkcjonalność automatycznego tłumaczenia tekstu z modułem przeznaczonym do detekcji emocji. Opracowane rozwiązanie obsługuje wiele języków oraz umożliwia prezentację rozkładu prawdopodobieństw dla poszczególnych klas emocji w analizowanym tekście, stanowiąc praktyczne narzędzie do eksperymentów oraz wizualizacji wyników w środowiskach produkcyjnych.

Rozdział 1

Podstawy przetwarzania języka naturalnego

Przetwarzanie języka naturalnego (ang. *Natural Language Processing*, NLP) stanowi jeden z kluczowych i najszybciej rozwijających się obszarów współczesnej sztucznej inteligencji. Jego celem jest umożliwienie maszynom analizy, rozumienia oraz generowania języka naturalnego w sposób zbliżony do ludzkiego. Postęp w tej dziedzinie znacząco wzmocnił rozwój interakcji między człowiekiem a komputerem, umożliwiając powstawanie systemów zdolnych do efektywnego przetwarzania tekstu i mowy. Dynamiczne zmiany w tym obszarze otwierają nowe możliwości dla zaawansowanych technologii, a jednocześnie znajdują coraz szersze zastosowanie w życiu codziennym.

1.1. Definicja NLP oraz jego znaczenie w obszarze sztucznej inteligencji

Przetwarzanie języka naturalnego jest interdyscyplinarną dziedziną łączącą elementy sztucznej inteligencji (ang. *Artificial Intelligence*, AI), informatyki, lingwistyki komputerowej i statystyki. W praktycznym zastosowaniu NLP rozwijane są zarówno modele teoretyczne, takie jak gramatyki formalne czy koncepcje semantyczne, a także rozwiązania o charakterze aplikacyjnym, do których należą między innymi klasyfikatory tekstu, systemy tłumaczenia automatycznego oraz chatboty. Gwałtowny przyrost danych tekstowych sprawił, że przetwarzanie języka naturalnego zyskało status kluczowej technologii umożliwiającej automatyczną analizę treści oraz projektowanie inteligentnych systemów wykorzystywanych w wielu obszarach życia [15].

Proces rozumienia języka naturalnego przez maszynę obejmuje szereg etapów przetwarzania tekstu wejściowego. Na początku wykonywane jest wstępne przygotowanie danych polegające na ich oczyszczeniu i normalizacji. Na tym etapie usuwane są znaki specjalne i interpunkcyjne, a wyrazy sprowadzane są do ich podstawowej formy, między innymi poprzez lematyzację. Następnie tekst poddawany jest tokenizacji, w ramach której dzielony jest na mniejsze jednostki językowe. Kolejnym krokiem jest analiza składniowa opierająca się na ustaleniu struktury zdania oraz zidentyfikowaniu

zależności pomiędzy poszczególnymi tokenami. Po niej następuje analiza semantyczna, która przypisuje znaczenia słowom i frazom, a także identyfikuje relacje między nimi, często z wykorzystaniem reguł lub modeli statystycznych. Informacje uzyskane na tym etapie łączone są z reprezentacją wektorową tekstu, stanowiącą podstawę dla klasycznych modeli NLP.

W nowoczesnych podejściach do NLP cały proces ulega uproszczeniu, ponieważ czyszczenie danych jest ograniczane w celu zachowania istotnych informacji semantycznych, a tokenizacja często przebiega równolegle z automatycznym modelowaniem kontekstu zdania. Na bazie reprezentacji wektorowej tekstu działają zaawansowane modele oparte na architekturze Transformer, takie jak BERT czy GPT, które są w stanie automatycznie interpretować znaczenie kontekstowe i wykonywać zadania takie jak klasyfikacja, tłumaczenie czy generacja tekstu.

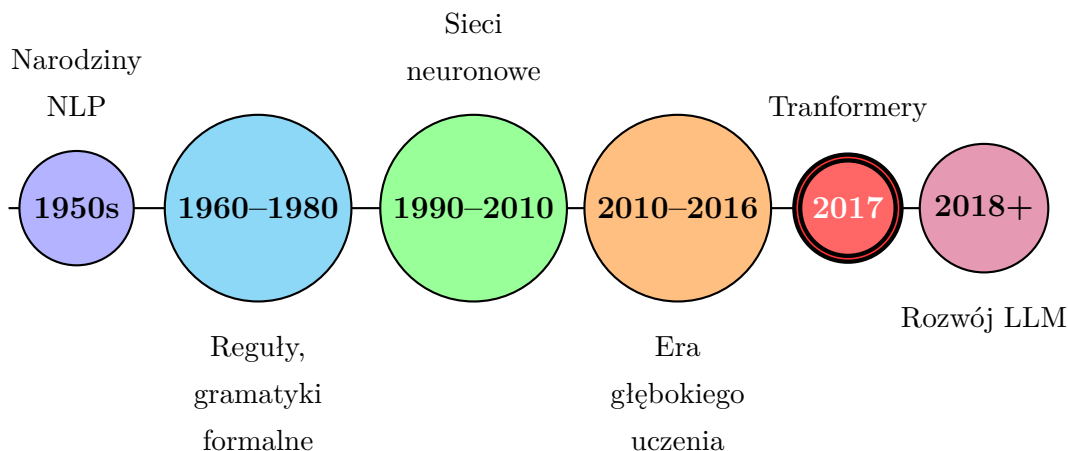
Pomimo znacznego postępu i rosnących możliwości współczesnych modeli językowych, ich sposób przetwarzania danych wciąż zasadniczo różni się od ludzkiego rozumienia języka. Ludzie zazwyczaj interpretują wypowiedzi dzięki bezpośredniemu doświadczeniu świata obejmującemu percepcję zmysłową oraz interakcję z otoczeniem, a słowa i pojęcia mają dla nich odniesienie do rzeczywistych przedmiotów, działań oraz emocji. Z kolei maszyny operują wyłącznie na symbolach i statystycznych zależnościach pomiędzy nimi, co sprawia, że ich „rozumienie” nie jest zakorzenione w rzeczywistości. Bez dostępu do danych sensorycznych lub kontekstu sytuacyjnego modele językowe nie dysponują pełnym obrazem świata takim jak ludzie [13]. Człowiek potrafi uczyć się i generalizować na podstawie niewielkiej liczby przykładów, łącząc nową informację z wcześniej zdobytą wiedzą i doświadczeniem. Natomiast modele NLP wymagają bardzo dużych zbiorów danych uczących, a ich zdolność do przenoszenia wiedzy między różnymi dziedzinami jest ograniczona. W celu zwiększenia elastyczności stosuje się techniki takie jak mechanizmy transferu wiedzy, jednak nadal nie dorównują one ludzkiej zdolności adaptacji poznawczej.

Podsumowując, obecne systemy NLP łączą analizę i generowanie języka w złożonych strukturach, które są wynikiem wielu dekad badań. Analiza etapów ich rozwoju pozwala zrozumieć w jaki sposób osiągnięto obecny poziom zaawansowania technologicznego, od pierwszych prób tłumaczenia maszynowego, przez epokę reguł i statystyki, aż po przełomowe rozwiązania oparte na głębokim uczeniu.

1.2. Ewolucja przetwarzania języka naturalnego

Rozwój przetwarzania języka naturalnego przebiegał etapami, które można podzielić na sześć głównych faz wynikających z postępu technologicznego, zmian w do-

stępności danych oraz w podejściu do modelowania języka. Zestawienie głównych etapów rozwoju NLP przedstawiono na Rysunku 1.1.



Rysunek 1.1: Ewolucja NLP - główne etapy rozwoju

Źródło: Opracowanie własne

Poniżej zaprezentowano szczegółowy opis kolejnych etapów ewolucji przetwarzania języka naturalnego:

Etap 1. Narodziny NLP (lata 50.)

Początki NLP związane są z okresem zimnej wojny, w którym pojawiła się strategiczna potrzeba automatycznego tłumaczenia tekstów, zwłaszcza z języka rosyjskiego na angielski. W 1954 roku przeprowadzono eksperyment, w którym komputer przetłumaczył 60 zdań z rosyjskiego na angielski, co wzbudziło ogromne nadzieje na szybki rozwój tłumaczenia maszynowego. W tym czasie dominowały metody lingwistyczne oparte na analizie składniowej i regułach gramatycznych.

Etap 2. Dominacja reguł i gramatyk formalnych (1960 – 1980)

Na tym etapie rozwój NLP opierał się na ręcznie tworzonych regułach, słownikach i formalnych gramatykach, co wymagało ogromnej pracy ekspertów językowych. W 1966 roku raport ALPAC wykazał ograniczenia ówczesnych metod, między innymi brak skalowalności i wysokie koszty, co doprowadziło do znaczącego ograniczenia finansowania w USA.

Etap 3. Metody statystyczne i uczenie maszynowe (1990 – 2010)

Rozwój mocy obliczeniowej i dostęp do większych korpusów umożliwiły przejście na metody probabilistyczne. Wprowadzono modele n -gramowe oraz ukryte modele Markowa (HMM), które zrewolucjonizowały rozpoznawanie mowy oraz automatyczne

tagowanie części zdania. Pod koniec tego okresu zaczęto eksperymentować z pierwszymi sieciami neuronowymi w NLP.

Etap 4. Głębokie uczenie (2010 – 2016)

Wprowadzenie głębokich sieci neuronowych, takich jak rekurencyjne sieci neuronowe (ang. *Recurrent Neural Network*, RNN), LSTM (ang. *Long Short-Term Memory*, LSTM) i GRU (ang. *Gated Recurrent Unit*, GRU), umożliwiło efektywne modelowanie sekwencji oraz długodystansowych zależności w tekście. Równolegle rozwijano również metody reprezentacji semantycznej słów, takie jak Word2Vec oraz GloVe, które stały się fundamentem dla późniejszych modeli kontekstowych.

Etap 5. Przełom architektury typu Transformer (2017)

Publikacja artykułu „Attention Is All You Need” w *Advances in Neural Information Processing Systems* [32] wprowadziła przełomową architekturę Transformer, która dzięki mechanizmowi uwagi (ang. *Attention Mechanism*) umożliwiła równoległe przetwarzanie sekwencji oraz modelowanie zależności w całym tekście. Zastosowane rozwiązanie wyeliminowało wiele ograniczeń charakterystycznych dla sieci rekurencyjnych i stało się podstawą rozwoju wszystkich późniejszych modeli NLP.

Etap 6. Era dużych modeli językowych (2018+)

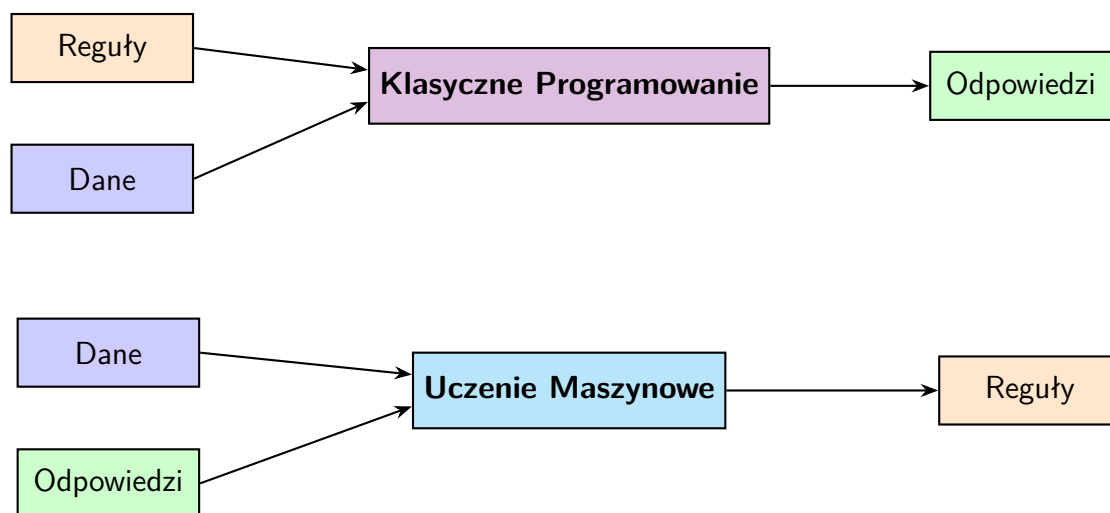
Na bazie architektury Transformer powstały modele takie jak: BERT, GPT-3, T5 oraz ich późniejsze warianty, które osiągają przełomowe wyniki w wielu zadaniach NLP. Upowszechniło się podejście oparte na uczeniu transferowym (ang. *Transfer Learning*) polegające na wykorzystaniu modeli wstępnie wytrenowanych na dużych korpusach. Równocześnie pojawiły się szerokie wdrożenia komercyjne (np. ChatGPT, Copilot, Gemini), co znacząco zmieniło sposób interakcji ludzi z technologią.

Podsumowując, ewolucja przetwarzania języka naturalnego obejmuje przejście od początkowych podejść opartych na regułach do metod wykorzystujących złożone modele statystyczne i głębokie sieci neuronowe. Kolejne etapy rozwoju prowadziły do zwiększania poziomu automatyzacji, skalowalności i zdolności do uchwycenia kontekstu językowego, co ostatecznie umożliwiło powstanie współczesnych dużych modeli językowych opartych na architekturze Transformer. Współczesne rozwiązania NLP bazują na tych osiągnięciach i łączą różnorodne techniki analizy języka, stanowiąc fundament zaawansowanych systemów dialogowych oraz aplikacji przetwarzania tekstu. Więcej informacji dotyczących poszczególnych etapów rozwoju NLP można znaleźć w [11, 13].

1.3. Rola sztucznej inteligencji w zadaniach przetwarzania języka naturalnego

Język naturalny charakteryzuje się ogromną złożonością i wieloznacznością. Słowa często przyjmują różne znaczenia w zależności od kontekstu, a w komunikacji powszechnie występują idiomy, metafory oraz ironia. Dodatkowym wyzwaniem jest wrażliwość języka na subtelne zmiany semantyczne, ponieważ nawet pojedynczy element, taki jak negacja („lubie” kontra „nie lubie”), potrafi całkowicie odmienić sens wypowiedzi. Ponadto język nieustannie ewoluuje, pojawiają się nowe słowa, a także zmienia się sposób ich użycia, co czyni go szczególnie trudnym do opisania za pomocą prostych, ręcznie tworzonych reguł [13].

Zastosowanie metod uczenia maszynowego (*ang. Machine Learning, ML*), zwłaszcza tych wykorzystujących uczenie głębokie (*ang. Deep Learning, DL*), całkowicie odmieniło sposób przetwarzania języka naturalnego. Modele ML oraz DL uczą się bezpośrednio z dużych zbiorów danych, co eliminuje konieczność ręcznego definiowania reguł i pozwala na automatyczne wykrywanie złożonych wzorców językowych [31]. Rysunek 1.2 przedstawia porównanie klasycznego programowania z uczeniem maszynowym prezentując różnice w przetwarzaniu danych i tworzeniu reguł.



Rysunek 1.2: Porównanie: klasyczne programowanie vs uczenie maszynowe

Źródło: Opracowanie własne

Kluczową rolę w przetwarzaniu języka naturalnego odgrywają głębokie sieci neuronowe, które uczą się reprezentacji wektorowych, zwanych osadzeniami słów (*ang. Embeddings*), umożliwiającymi odwzorowywanie znaczenia słów wraz z ich kontekstem. Osadzenia słów zrewolucjonizowały reprezentację i przetwarzanie tekstu pozwalając na efektywną transformację danych tekstowych na postać numeryczną, a także

na dynamiczne i kontekstowe odwzorowanie znaczeń i relacji między wyrazami, co znacząco zwiększyło precyzję działania modeli NLP. Obecnie większość nowoczesnych systemów NLP wykorzystuje osadzenia słów, których jakość w dużej mierze decyduje o skuteczności modeli. Zależy ona między innymi od wielkości i różnorodności korpusu treningowego oraz od zdolności modelu do uchwycenia kontekstu i zależności między wyrazami. Osadzenia kontekstowe generowane przez modele takie jak BERT czy GPT zdecydowanie przewyższają tradycyjne osadzenia statyczne, ponieważ dynamicznie dostosowują reprezentację słowa do jego otoczenia w zdaniu [27].

W klasycznym programowaniu reguły są definiowane przez eksperta i wraz z danymi przekazywane do programu, który na ich podstawie generuje odpowiedzi. Takie programy są w pełni interpretowalne, lecz mało skalowalne i trudne w utrzymaniu. Wraz ze wzrostem złożoności analizowanych danych podejście oparte na sztywnych regułach szybko okazało się niewystarczające, ponieważ nie wykazywało zdolności do uogólniania wiedzy ani adaptowania się do nowych form językowych. Każda modyfikacja wymagała ręcznej ingerencji ekspertów, przez co rozwój i utrzymanie takich rozwiązań stawało się kosztowne oraz podatne na błędy [13, 31].

Rozwój i ewolucja metod przetwarzania języka naturalnego doprowadziły do powstania nowych podejść opartych na uczeniu maszynowym. W odróżnieniu od klasycznych systemów regułowych, modele te uczą się bezpośrednio z danych, automatycznie identyfikując wzorce i zależności w języku. Dzięki temu umożliwiają bardziej elastyczne i skalowalne rozwiązania, które potrafią uogólniać wiedzę oraz adaptować się do nowych kontekstów językowych. W szczególności, architektura typu Transformer umożliwiła analizę całej sekwencji słów równocześnie, co pozwoliło na skuteczne modelowanie zależności występujących nawet między odległymi elementami sekwencji. W konsekwencji, modele takie jak BERT czy GPT znacząco podnoszą skuteczność zadań NLP dzięki precyzyjnemu uchwyceniu kontekstu, tworzeniu bogatych reprezentacji słów oraz zwiększonej zdolności generalizacji [31].

W praktyce często wykorzystuje się systemy hybrydowe, które łączą podejścia regułowe z metodami statystycznymi oraz głębokim uczeniem. Reguły lingwistyczne służą tu przede wszystkim do wstępnego przetwarzania tekstu czy filtrowania danych. Tego typu rozwiązania znajdują zastosowanie zwłaszcza w dziedzinach niszowych, w których występuje rzadkie lub specjalistyczne słownictwo. W takich przypadkach modele głębokiego uczenia mogą nie dysponować wystarczającym kontekstem do poprawnej interpretacji słów, dlatego wspomaganie ich wyszukiwaniem regułowym po słowach kluczowych pozwala skuteczniej wychwytywać istotne informacje i poprawiać jakość analizy tekstu. W niektórych sytuacjach rozwiązania oparte wyłącznie na regułach są w pełni wystarczające, szczególnie w mniej złożonych zadaniach, takich

jak wykrywanie spamu czy proste klasyfikacje tematyczne. Systemy hybrydowe stają się jednak coraz bardziej popularne, a jednym z kluczowych kierunków badań jest opracowanie metod pozwalających na skuteczne i optymalne integrowanie wyników uzyskiwanych z różnych modeli w celu maksymalizacji jakości końcowych rezultatów, por. [31].

Ewolucja sztucznej inteligencji znacząco rozszerzyła zakres analizowanych danych, umożliwiając przetwarzanie nie tylko tekstu, lecz także obrazu i dźwięku. W przypadku danych dźwiękowych kluczowym krokiem jest transkrypcja na tekst, czyli zamiana mowy na formę pisaną, co umożliwia stosowanie standardowych metod NLP. Ponadto przetwarzanie obrazu znalazło zastosowanie między innymi w diagnostyce medycznej. Systemy te analizują obrazy medyczne, a następnie generują do nich opis w języku naturalnym, co wspomaga lekarzy w interpretacji wyników i podejmowaniu decyzji klinicznych. W efekcie coraz większą popularność zyskują rozwiązania multimodalne, które integrują różne rodzaje danych w ramach jednej architektury, umożliwiając tworzenie bardziej złożonych i adaptacyjnych systemów sztucznej inteligencji zdolnych do pracy w realistycznych, wieloaspektowych scenariuszach.

Rozdział 2

Metody reprezentacji tekstu w NLP

Reprezentacja tekstu jest kluczowym elementem w przetwarzaniu języka naturalnego. Umożliwia komputerom zrozumienie i analizę ludzkiego języka poprzez zamianę słów na postać numeryczną. Pierwsze metody reprezentacji tekstu opierały się na prostych regułach statystycznych, które zamieniały tekst na wektory odzwierciedlające obecność lub częstotliwość występowania słów w dokumentach. Chociaż metody te cechowały się prostotą i sprawdzały się w podstawowych zastosowaniach, ich użyteczność była ograniczona z uwagi na nieuwzględnianie struktury składniowej oraz niezdolność do uchwycenia kontekstu semantycznego. Następnie pojawiły się statyczne osadzenia słów, które pozwoliły na uchwycenie podobieństw semantycznych między słowami w przestrzeni wektorowej. Przełom w dziedzinie przetwarzania języka naturalnego nastąpił wraz z pojawieniem się kontekstowych osadzeń słów, opartych na architekturze Transformer. Osadzenia te umożliwiły dynamiczne tworzenie wektorowych reprezentacji, co znacznie poprawiło jakość przeprowadzanych analiz w nowoczesnych systemach NLP.

2.1. Klasyczne metody reprezentacji tekstu

Jedną z najbardziej klasycznych metod reprezentacji tekstu jest model worka słów (ang. *Bag-of-Words*, BoW), w którym dokument jest reprezentowany jako wektor liczb odzwierciedlających pojawienie się danego terminu lub jego częstość, bez uwzględniania kolejności słów. Ograniczeniem tego podejścia jest generowanie rzadkich, wysokowymiarowych wektorów, w których większość współrzędnych ma wartość równą 0, co znacząco utrudnia ich porównywanie [33]. W wariantcie binarnym modelu BoW, uwzględniającym jedynie obecność słów w tekście, dokument d reprezentuje się jako wektor o długości V , odpowiadający liczbie unikalnych terminów w słowniku, o następującej postaci

$$\mathbf{x}_d = (x_{1,d}, x_{2,d}, \dots, x_{V,d}),$$

gdzie $\mathbf{x}_d$ jest wektorem reprezentującym dokument d w przestrzeni cech, zaś V stanowi liczbę wszystkich unikalnych słów określającą rozmiar słownika. Element $x_{i,d}$ informuje czy i -te słowo wystąpiło w dokumencie d (0 - jeśli nie wystąpiło, 1 - jeśli wystąpiło), zaś $i = 1, 2, \dots, V$ oznacza indeks danego słowa w słowniku. Wariant zliczający BoW reprezentuje liczbę wystąpień słowa w dokumencie, przy czym najczęściej stosowaną miarą jest surowa częstotliwość terminu (ang. *Term Frequency*, TF)

$$x_{t,d} = f_{t,d},$$

gdzie $f_{t,d}$ to liczba wystąpień terminu t w dokumencie d .

Model BoW jest prosty w implementacji i zapewnia w pełni interpretowalną reprezentację tekstu. Dodatkowo jest rozwiązaniem efektywnym pamięciowo, co ma szczególne znaczenie przy analizie dużych korpusów. Jednocześnie wysoka wymiarowość tworzonych wektorów może prowadzić do klątwy wymiarowości, zwiększać ryzyko przeuczenia modeli, a także ograniczać zdolność do uchwycenia subtelnych różnic pomiędzy tekstami. Ponadto często występujące słowa, takie jak spójniki czy inne elementy funkcjonalne języka, mogą nadmiernie kształtować uzyskaną reprezentację, mimo że nie wnoszą istotnej wartości informacyjnej. Ze względu na te ograniczenia opracowano bardziej zaawansowane metody analizy tekstu.

Rozszerzeniem modelu BoW jest schemat ważenia terminów TF-IDF (ang. *Term Frequency-Inverse Document Frequency*), którego celem jest ograniczenie wpływu słów występujących bardzo często w korpusie, takich jak „i” czy „że”, które nie dostarczają istotnej wartości informacyjnej. Wartość TF rozszerza się o wagę odwrotnej częstości dokumentowej IDF, obniżającej znaczenie zbyt popularnych słów. Metoda ta jest szybka i łatwa do obliczenia oraz uwzględnia różną długość dokumentów. Jednocześnie nie odzwierciedla ona semantyki słów, gdyż opiera się wyłącznie na statystycznej analizie częstości wystąpień. Dla terminu t w dokumencie d i całym korpusie D , wartość TF-IDF definiowana jest następująco

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D),$$

gdzie $\text{TF}(t, d)$ jest częstością występowania terminu t w dokumencie d , zaś $\text{IDF}(t, D)$ oznacza odwrotną częstość występowania terminu t w całym korpusie D , która wyrażona jest wzorem

$$\text{IDF}(t, D) = \log \left(\frac{|D|}{1 + |\{d \in D : t \in d\}|} \right).$$

Połączenie składowej TF, opisującej istotność terminu w pojedynczym dokumencie, z wagą IDF, odzwierciedlającą jego rozpowszechnienie w całym korpusie,

umożliwia wyznaczenie wag preferujących słowa charakterystyczne dla konkretnego dokumentu przy jednoczesnym ograniczeniu wpływu terminów powszechnych [35].

Schemat TF-IDF stanowi praktyczną i często skuteczną metodę reprezentacji tekstu, która dzięki uwzględnieniu informacji globalnej o rozkładzie terminów w korpusie przewyższa klasyczny model BoW. Niemniej jednak TF-IDF nie rozwiązuje problemu semantyki ani wieloznaczności słów, a uzyskane wektory pozostają rzadkie i wysokowymiarowe. Ponadto waga IDF jest wrażliwa na wielkość korpusu i w przypadku niewielkich zbiorów danych może prowadzić do nadmiernego wzmocnienia terminów występujących wyłącznie w pojedynczych dokumentach. TF-IDF nie uwzględnia również kolejności słów, a wyznaczanie składowej IDF wiąże się z większym kosztem obliczeniowym w porównaniu z klasycznym modelem BoW [33, 35].

Jednym z pierwszych modeli stosowanych w analizie tekstu, które pozwalają na częściowe uwzględnienie kolejności wyrazów oraz lokalnego kontekstu językowego, są podejścia oparte na n -gramach. Modele n -gramowe definiuje się jako ciągi n -kolejnych słów, gdzie unigramy odpowiadają pojedynczym słowom, bigramy parom, a trigramy trójkom wyrazów. Utworzenie modelu n -gramowego rozpoczyna się od zliczania wystąpień sekwencji o ustalonej długości n w istniejących zasobach językowych. Model BoW rozszerzony o n -gramy traktuje każdy n -gram jako osobny termin. Dla korpusu oraz słownika unigramów o rozmiarze $|V_1|$, liczba możliwych n -gramów rzędu n rośnie asymptotycznie jak $|V_1|^n$. W praktycznych zastosowaniach najczęściej ogranicza się analizę do unigramów i bigramów, gdyż dalsze zwiększanie wartości n prowadzi do gwałtownego wzrostu wymiarowości przestrzeni cech oraz nasilenia problemu rzadkości danych [13, 20].

Model językowy oparty na n -gramach przybliża rozkład sekwencji słów przez ograniczenie zależności do ostatnich $n - 1$ słów. Z reguły łańcucha (ang. *Chain Rule*) wynika, że

$$P(w_1, \dots, w_m) = \prod_{i=1}^m P(w_i \mid w_1, \dots, w_{i-1}),$$

gdzie w_i oznacza i -te słowo w sekwencji, o długości m -słów, $P(w_1, \dots, w_m)$ jest prawdopodobieństwem zaobserwowania sekwencji $w_1, \dots, w_m$, zaś $P(w_i \mid w_1, \dots, w_{i-1})$ to warunkowe prawdopodobieństwo wystąpienia słowa w_i na podstawie wszystkich poprzedzających słów w sekwencji $(w_1, \dots, w_{i-1})$. Rozkład ten jest dokładny, jednak jego bezpośrednia estymacja w praktyce okazuje się mało skuteczna, ponieważ wraz ze wzrostem długości sekwencji gwałtownie rośnie liczba możliwych historii kontekstowych.

W modelach n -gramowych wprowadza się zatem przybliżenie Markowa rzędu $n - 1$, zakładając, że prawdopodobieństwo wystąpienia danego słowa zależy wyłącznie

od $n - 1$ bezpośrednio poprzedzających je wyrazów i wyrażone jest wzorem

$$P(w_1, \dots, w_m) \approx \prod_{i=1}^m P(w_i \mid w_{i-n+1}, \dots, w_{i-1}),$$

gdzie n oznacza rząd n -gramu. Przybliżenie to ogranicza zakres kontekstu do co najwyżej $n - 1$ parametrów modelu [13, 20]. Parametry estymuje się zwykle przez estymator największej wiarygodności (ang. *Maximum Likelihood Estimation*, MLE) postaci

$$P(w_i \mid c) = \frac{\text{count}(c, w_i)}{\text{count}(c)},$$

gdzie $c = (w_{i-n+1}, \dots, w_{i-1})$ reprezentuje kontekst (historyczny n -gram bez słowa docelowego), $\text{count}(c, w_i)$ oznacza liczbę wystąpień sekwencji (c, w_i) w korpusie, zaś $\text{count}(c)$ to liczba wystąpień samego kontekstu c .

Zastosowanie MLE prowadzi do przyporządkowywania zerowych prawdopodobieństw n -gramom nieobserwowanym w zbiorze uczącym, co stanowi jedno z głównych ograniczeń modeli n -gramowych. W praktyce skutkuje to nieadekwatnym modelowaniem języka i niską zdolnością do generalizacji. W celu rozwiązania tego problemu stosuje się metody wygładzania, które umożliwiają przypisanie dodatnich prawdopodobieństw rzadkim oraz niezaobserwowanym n -gramom, poprawiając tym samym jakość estymowanego rozkładu. Modele n -gramowe częściowo uchwytyją lokalny kontekst i frazy, co poprawia ich zdolność do reprezentacji znaczenia tekstu. Jednocześnie prowadzą one do wzrostu liczby parametrów modelu, a tym samym do zwiększenia wymiarowości słownika. W konsekwencji nasila się problem rzadkości danych oraz kłątwa wymiarowości, co znacząco utrudnia przetwarzanie danych tekstowych [13].

Podsumowując, klasyczne reprezentacje tekstu posiadają istotne ograniczenia wpływające na efektywność uczenia oraz jakość predykcji modeli. Ograniczenia te wynikają przede wszystkim z wysokiej wymiarowości i rzadkości reprezentacji oraz braku jawnego modelowania semantyki oraz kontekstu językowego. W rzeczywistości wszystkie podejścia oparte na statystyce można traktować jako „ważone” formy BoW, w których proces wektoryzacji tekstu ma charakter deterministyczny, co stanowi istotną zaletę w postaci pełnej wyjaśnialności generowanych cech. Ostatecznie, klasyczne reprezentacje tekstu pozostają fundamentem wielu systemów przetwarzania języka naturalnego oraz stanowią naturalny punkt wyjścia dla bardziej zaawansowanych rozwiązań [35].

2.2. Statyczne metody reprezentacji tekstu

Kolejnym podejściem stosowanym w przetwarzaniu języka naturalnego są statyczne metody reprezentacji tekstu. Statyczne reprezentacje osadzeń słów przypisują

każdemu tokenowi w jeden stały wektor liczbowy $\mathbf{v}(w) \in \mathbb{R}^d$, który pozostaje niezmienny niezależnie od kontekstu w jakim dane słowo występuje. Oznacza to, że przykładowo słowo „zamek” występujące w znaczeniu budowli, mechanizmu w drzwiach czy zamka błyskawicznego, będzie miało zawsze taką samą reprezentację wektorową. W konsekwencji, statyczne osadzenia słów można formalnie opisać jako funkcję

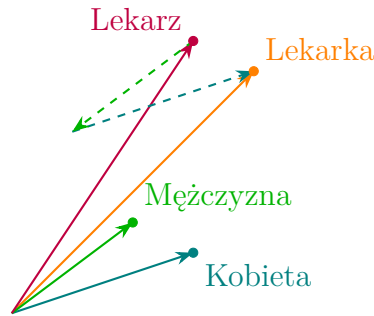
$$f : V \longrightarrow \mathbb{R}^d,$$

gdzie V jest słownikiem języka, zaś d jest wymiarem przestrzeni wektorowej. Uzyskane reprezentacje mają charakter gęsty, przy czym $d \ll |V|$, a wszystkie słowa są osadzone we wspólnej przestrzeni semantycznej. Podobieństwo pomiędzy słowami może być wyznaczane geometrycznie z wykorzystaniem różnych miar odległości i podobieństwa w przestrzeni wektorowej. W przypadku osadzeń słów najczęściej stosowaną miarą jest cosinus kąta pomiędzy wektorami

$$\cos(\mathbf{v}(w_i), \mathbf{v}(w_j)) = \frac{\mathbf{v}(w_i) \cdot \mathbf{v}(w_j)}{\|\mathbf{v}(w_i)\| \|\mathbf{v}(w_j)\|}.$$

W dobrze zdefiniowanych przestrzeniach semantycznych słowa o podobnym znaczeniu są reprezentowane przez wektory położone blisko siebie, natomiast słowa o odmiennym znaczeniu wykazują niski stopień podobieństwa wektorowego. Dodatkowo operacje algebraiczne wykonywane na wektorach osadzeń często zachowują relacje odległościowe w przestrzeni wektorowej [13, 29]. Rysunek 2.1 przedstawia przykładowe relacje pomiędzy słowami w przestrzeni osadzeń słów.

$$\mathbf{v}(\text{Lekarz}) - \mathbf{v}(\text{Mężczyzna}) + \mathbf{v}(\text{Kobieta}) \approx \mathbf{v}(\text{Lekarka})$$



Rysunek 2.1: Przykład relacji wektorowych w przestrzeni statycznych osadzeń słów

Źródło: Opracowanie własne na podstawie [13, 21, 25]

Jedną z pierwszych oraz najbardziej rozpowszechnionych statycznych metod reprezentacji tekstu jest model Word2Vec. Model ten uczy się relacji między słowami na podstawie ich otoczenia za pomocą mapowania słów do gęstych wektorów w $\mathbb{R}^d$,

gdzie d jest z góry ustaloną liczbą wymiarów przestrzeni wektorowej [21, 34]. Model Word2Vec ma dwie podstawowe architektury:

i) model CBoW (ang. *Continuous Bag-of-Words*, CBoW) przewidujący słowo centralne w_t na podstawie uśrednienia wektorów słów kontekstowych występujących w oknie kontekstu o rozmiarze c . W tym celu minimalizowany jest logarytm prawdopodobieństwa wystąpienia słowa centralnego przy zadanym kontekście

$$-\sum_{t=1}^T \log P(w_t \mid w_{t-c}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+c}),$$

gdzie T oznacza liczbę tokenów w korpusie, w_t reprezentuje słowo w pozycji t , zaś c to rozmiar okna kontekstowego,

ii) model Skip-gram realizujący predykcję słów kontekstowych na podstawie słowa centralnego w_t . Dla każdego wystąpienia słowa w_t maksymalizowane jest średnie logarytmiczne prawdopodobieństwo pojawienia się słów wchodzących w skład jego kontekstu. Formalnie można zapisać to następująco

$$\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-c \leq j \leq c \\ j \neq 0}} \log P(w_{t+j} \mid w_t).$$

Model Skip-gram jest bardziej kosztowny obliczeniowo w porównaniu do modelu CBoW, natomiast zazwyczaj osiąga lepsze wyniki w zadaniach wymagających modelowania relacji semantycznych. Zarówno w architekturze CBOW, jak i Skip-gram, prawdopodobieństwo wystąpienia słowa centralnego lub słów należących do jego kontekstu obliczane jest z wykorzystaniem funkcji softmax ze wzoru

$$P(w_O \mid w_I) = \frac{\exp(\mathbf{u}_{w_O}^\top \mathbf{v}_{w_I})}{\sum_{w \in V} \exp(\mathbf{u}_w^\top \mathbf{v}_{w_I})},$$

gdzie $\mathbf{v}_{w_I}$ oznacza wektor wejściowy słowa w_I , $\mathbf{u}_{w_O}$ reprezentuje wektor wyjściowy słowa w_O , zaś $\sum_{w \in V}$ to sumowanie po wszystkich słowach słownika V . Obie architektury uczą macierze wag $W \in \mathbb{R}^{|V| \times d}$, gdzie wiersz W_i jest wektorem słowa i . Po trenin-gu uzyskiwane są stałe reprezentacje wektorowe osadzeń słów $\mathbf{v}_w \in \mathbb{R}^d$. Ostatecznie, element $P(w_O \mid w_I)$ wyznaczany jest przez warstwę softmax [21, 22, 34].

Kolejnym modelem statycznych reprezentacji tekstu jest GloVe (ang. *Global Vectors for Word Representation*, GloVe), który wykorzystuje globalne statystyki współwystępowania słów do uczenia reprezentacji wektorowych. W przeciwieństwie do modelu Word2Vec, który opiera się na lokalnym kontekście (na oknach słów), GloVe bazuje na macierzy współwystąpień całego korpusu [25].

Niech $X \in \mathbb{R}^{|V| \times |V|}$ oznacza macierz współwystępowania. Dla każdego słowa w_i definiuje się wartość

$$X_i = \sum_j X_{ij},$$

która oznacza całkowitą liczbę wystąpień słów w kontekście słowa w_i , natomiast X_{ij} określa liczbę wystąpień słowa w_j w otoczeniu słowa w_i . Model GloVe bazuje na założeniu, że stosunki prawdopodobieństw warunkowych współwystępowania słów odzwierciedlają relacje semantyczne. Dla słów w_i, w_j oraz słowa kontekstowego w_k zależność tę opisuje równanie

$$\frac{P_{ik}}{P_{jk}} = \frac{X_{ik}/X_i}{X_{jk}/X_j},$$

gdzie P_{ik} oznacza prawdopodobieństwo wystąpienia słowa w_k w kontekście słowa w_i . Na tej podstawie zdefiniowano funkcję celu, która aproksymuje logarytm liczby współwystąpień danego słowa

$$\mathbf{u}_i^\top \mathbf{v}_j + b_i + b_j \approx \log X_{ij},$$

gdzie $\mathbf{u}_i \in \mathbb{R}^d$ oznacza wektor słowa w_i , $\mathbf{v}_j \in \mathbb{R}^d$ reprezentuje wektor kontekstu słowa w_j , zaś b_i, b_j to błędy losowe. Ostateczna postać funkcji celu to ważona regresja kwadratowa

$$J = \sum_{i,j=1}^{|V|} f(X_{ij}) \left(\mathbf{u}_i^\top \mathbf{v}_j + b_i + b_j - \log X_{ij} \right)^2,$$

gdzie $f(x)$ to funkcja skalująca ograniczająca wpływ bardzo częstych par, definiowana jako

$$f(x) = \begin{cases} (x/x_{\max})^\alpha & \text{dla } x < x_{\max}, \\ 1 & \text{dla } x \geq x_{\max}, \end{cases}$$

gdzie $\alpha \in (0, 1)$ oraz $x_{\max}$ oznacza próg częstości. W wyniku optymalizacji otrzymywane są wektory, które odtwarzają zarówno relacje semantyczne, jak i syntaktyczne. Ostatecznie, model GloVe dzięki globalnemu podejściu uwzględniającemu wszystkie pary współwystąpień zamiast jedynie lokalnych okien kontekstowych, pozwala lepiej wykorzystać pełną statystykę języka. Dodatkowo pozostaje obliczeniowo efektywny oraz łatwy do równoległej implementacji [25].

Innym podejściem stosowanym w ramach statycznych modeli osadzania słów jest fastText, zaproponowany przez Facebook AI Research, który rozwija idee modeli Word2Vec oraz GloVe poprzez uwzględnienie wewnętrznej struktury słów. W przeciwieństwie do klasycznych podejść, przypisujących pojedynczą reprezentację wektorową całemu wyrazowi, fastText modeluje je z wykorzystaniem reprezentacji fragmentów w postaci znakowych n -gramów [3, 12, 21].

Niech $\mathcal{G}(w)$ oznacza zbiór n -gramów słowa w (z uwzględnieniem znaczników początku i końca, np. trigramy słowa *jabłko*: $\langle \text{ja}, \text{jab}, \text{abł}, \text{błk}, \text{łko}, \text{ko} \rangle$). Reprezentacja słowa wyznaczana jest jako suma wektorów n -gramów

$$\mathbf{z}_w = \sum_{g \in \mathcal{G}(w)} \mathbf{z}_g,$$

gdzie $\mathbf{z}_g \in \mathbb{R}^d$ oznacza osadzenie n -gramu g . Zastosowanie takiej reprezentacji wprowadza szereg istotnych zalet w procesie osadzania słów. Po pierwsze, możliwa jest obsługa słów spoza słownika, ponieważ wektor słowa można skonstruować na podstawie jego fragmentów, nawet jeśli nie pojawiło się ono w korpusie treningowym. Po drugie, wykorzystanie znakowych n -gramów pozwala modelowi skuteczniej uchwycić informacje morfologiczne oraz uwzględniać złożoność fleksji językowej, na przykład różne formy tego samego słowa. Po trzecie, mimo rozszerzonego słownika n -gramów, model fast-Text zachowuje dobrą skalowalność i efektywność obliczeniową, również w przypadku uczenia na bardzo dużych zbiorach danych tekstowych [3, 12, 21].

Wszystkie opisane powyżej modele należą do grupy statycznych osadzeń słów, w których każde słowo otrzymuje jedną, niezmienną reprezentację wektorową niezależnie od kontekstu zdania. Model Word2Vec uczy wektory w oparciu o lokalne konteksty, GloVe wykorzystuje globalne statystyki współwystępowania słów, a FastText rozszerza Word2Vec o n -gramy znakowe, co umożliwi lepsze modelowanie zjawisk morfologicznych oraz skuteczniejszą obsługę słów rzadkich. Brak jawnego modelowania znaczeń zależnych od kontekstu stanowił jednak jeden z głównych impulsów do rozwoju współczesnych modeli językowych opartych na osadzeniach kontekstowych [3, 21, 25].

2.3. Kontekstowe metody reprezentacji tekstu

Kontekstowe metody reprezentacji osadzeń słów przypisują temu samemu słowu odmienne wektory w zależności od jego otoczenia językowego. Kluczowym przełomem, który umożliwił ich główny rozwój, było wprowadzenie architektury typu Transformer. W przeciwieństwie do wcześniejszych modeli, Transformer opiera się na mechanizmie uwagi pozwalającym modelować zależności pomiędzy dowolnymi słowami w zdaniu, niezależnie od ich wzajemnej odległości [32]. Należy jednak zaznaczyć, że pomimo kluczowej roli wspomnianej architektury, historycznie pierwszym modelem umożliwiającym generowanie kontekstowych osadzeń słów był ELMo (ang. *Embeddings from Language Models*).

ELMo jest modelem generującym kontekstowe reprezentacje osadzeń słów jako funkcje wewnętrznych stanów dwuwarstwowego dwukierunkowego modelu językowego (ang. *bidirectional Language Model*, biLM), umożliwiając otrzymanie różnych re-

prezentacji dla tego samego tokenu w zależności od otaczającego kontekstu [26]. Dla sekwencji tokenów $(t_1, \dots, t_N)$, gdzie N oznacza liczbę tokenów w sekwencji (długość analizowanego tekstu wyrażoną w jednostkach rozpoznawanych przez model), można zdefiniować następujące rozkłady:

i) model językowy w przód (ang. *Forward Language Model*) postaci

$$p_{\text{fwd}}(t_1, t_2, \dots, t_N) = \prod_{k=1}^N p(t_k | t_1, t_2, \dots, t_{k-1})$$

gdzie $t_1, t_2, \dots, t_N$ to tokeny w sekwencji (np. słowa w zdaniu), $p_{\text{fwd}}(\cdot)$ reprezentuje rozkład prawdopodobieństwa dla modelu językowego w przód, zaś $p(t_k | t_1, \dots, t_{k-1})$ oznacza prawdopodobieństwo, że kolejnym tokenem będzie t_k , gdy znane są wszystkie wcześniejsze tokeny $t_1, \dots, t_{k-1}$. Jest to klasyczne podejście w modelach językowych forward, które przewidują następne tokeny na podstawie kontekstu z lewej strony,

ii) model językowy wstecz (ang. *Backward Language Model*) postaci

$$p_{\text{bwd}}(t_1, t_2, \dots, t_N) = \prod_{k=1}^N p(t_k | t_{k+1}, t_{k+2}, \dots, t_N)$$

gdzie $p_{\text{bwd}}(\cdot)$ reprezentuje rozkład prawdopodobieństwa dla modelu językowego wstecz, $p(t_k | t_{k+1}, \dots, t_N)$ oznacza prawdopodobieństwo, że token t_k występuje, gdy znany jest jego przyszły kontekst (wszystkie słowa po nim). W tym podejściu model przewiduje następne tokeny na podstawie kontekstu z prawej strony.

Oba modele językowe różnią się kierunkiem, w którym uczą się modelować język. Model forward uczy się przewidywać słowa w kolejności naturalnej, czyli od lewej do prawej strony tekstu, natomiast model backward robi to w kierunku przeciwnym. Połączenie tych dwóch podejść w modelu biLM pozwala na stworzenie reprezentacji słowa uwzględniającej pełny kontekst sekwencji. Celem treningowym modelu biLM jest maksymalizacja łącznej logarytmicznej funkcji wiarygodności obu kierunków, co umożliwia efektywne kodowanie informacji kontekstowej w reprezentacji słowa:

$$\sum_{k=1}^N \left(\log p(t_k | t_1, \dots, t_{k-1}; \Theta_x, \vec{\Theta}_{\text{LSTM}}, \Theta_s) + \log p(t_k | t_{k+1}, \dots, t_N; \Theta_x, \overleftarrow{\Theta}_{\text{LSTM}}, \Theta_s) \right),$$

gdzie Θ_x to parametry warstwy wejściowej, $\vec{\Theta}_{\text{LSTM}}$ to parametry LSTM dla kierunku w przód, $\overleftarrow{\Theta}_{\text{LSTM}}$ to parametry LSTM dla kierunku w tył, Θ_s to parametry warstwy softmax, zaś N to długość sekwencji. Końcowa reprezentacja tokenu k w zadaniu *task* jest ważoną kombinacją wszystkich poziomów reprezentacji biLM i wyraża się wzorem

$$\text{ELMo}_k^{(\text{task})} = \gamma^{(\text{task})} \sum_{j=0}^L s_j^{(\text{task})} h_{k,j}^{\text{LM}},$$

gdzie $s^{(task)} = \text{softmax}(w^{(task)})$ oznacza uczone wagi warstw, $\gamma^{(task)} > 0$ reprezentuje uczony współczynnik skali, zaś $h_{k,j}^{LM}$ to reprezentacja z warstwy j . Dzięki takiemu podejściu osadzenia słów uwzględniają całe zdanie, co umożliwia skuteczne rozróżnianie znaczeń polisemicznych. Ostatecznie, poprzez swoją dwukierunkowość, model ELMo znacząco przewyższył dotychczasowe metody oparte na statycznych osadzeniach słów, osiągając lepsze wyniki w szerokim zakresie zadań przetwarzania języka naturalnego [26].

Pomimo wspomnianych osiągnięć dalszy rozwój metod reprezentacji języka wymagał jednak bardziej elastycznych i skutecznych rozwiązań. Odpowiedzią na te potrzeby było wprowadzenie w pracy [32] architektury typu Transformer, która umożliwiła nowe podejście do analizy języka, oparte na jednoczesnym przetwarzaniu całych sekwencji. Na tej podstawie powstały modele takie jak: BERT, RoBERTa czy GPT, które wyznaczyły nowe standardy jakości w wielu zadaniach przetwarzania tekstu. Fundamentalnym elementem tej architektury jest mechanizm uwagi (ang. *self-attention*) opierający się na skalowanej funkcji iloczynu skalarnego (ang. *scaled dot-product attention*). Dla danej sekwencji wejściowej wyznaczane są trzy macierze: zapytań (ang. *queries*) $Q \in \mathbb{R}^{n \times d_k}$, kluczy (ang. *keys*) $K \in \mathbb{R}^{n \times d_k}$ oraz wartości (ang. *values*) $V \in \mathbb{R}^{n \times d_v}$, gdzie n oznacza liczbę tokenów, d_k jest wymiarem przestrzeni zapytań i kluczy, a d_v odpowiada wymiarowi przestrzeni wartości. Zależność definiująca mechanizm uwagi ma postać

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

Macierz podobieństw między wszystkimi parami tokenów uzyskuje się przez iloczyn $QK^\top$, a następnie wyniki są skalowane przez $\sqrt{d_k}$, co pomaga stabilizować gradienty przy dużej liczbie wymiarów. Funkcja softmax normalizuje wartości do rozkładu prawdopodobieństwa, przypisując każdemu tokenowi odpowiednie wagi względem pozostałych. Ostatecznie, mnożenie przez V daje ważoną sumę wartości, która stanowi końcową reprezentację uwzględniającą kontekst wszystkich tokenów. W praktyce stosuje się wielogłową atencję (ang. *Multi-head Attention*), definiowaną jako

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O,$$

gdzie

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V),$$

przy czym W_i^Q, W_i^K, W_i^V oznaczają macierze wag przekształcające przestrzeń wejściową do podprzestrzeni o wymiarze d_k , zaś macierz $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ to macierz wyjściowa, która liniowo transformuje połączenie wszystkich głów atencji (h głów o wymiarze

d_v) z powrotem do pełnego wymiaru modelu d_{model} , scalając informacje z różnych podprzestrzeni uwagi w jedną spójną reprezentację [32].

W dekodерze architektury Transformer stosowany jest mechanizm maskowanej uwagi, którego celem jest uniemożliwienie modelowi wykorzystywania informacji z przyszłych pozycji w sekwencji podczas generowania kolejnych tokenów. Mechanizm maskowania realizowany jest poprzez dodanie do macierzy podobieństw $QK^\top$ macierzy maskującej M , której elementy o wartości $-\infty$ odpowiadają połączeniom niedozwolonym. W efekcie, po normalizacji za pomocą funkcji softmax, przypisane im wagi przyjmują wartość zero, co uniemożliwia uwzględnienie przyszłych pozycji w sekwencji. Formalnie mechanizm ten można zapisać w postaci

$$\text{MaskedAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top + M}{\sqrt{d_k}}\right) V,$$

gdzie M jest macierzą maskującą z wartościami 0 (dopuszczone połączenia) lub $-\infty$ (zablokowane połączenia). Dzięki temu model podczas generacji tekstu nie wykorzystuje informacji o przyszłych tokenach, a jedynie operuje na danych dostępnych na danym etapie generacji. Architektura Transformer składa się zatem z dwóch głównych części: enkodera, otrzymującego sekwencję źródłową i generującego jej reprezentację wektorową oraz dekodera, który na podstawie wyjścia enkodera, z wykorzystaniem mechanizmu maskowanej uwagi, generuje sekwencję wynikową token po tokenie [32].

Pomimo wcześniejszej realizacji kontekstowych osadzeń słów w modelu ELMo, architektura Transformer umożliwiła ich bardziej efektywne i skalowalne modelowanie, co przyczyniło się do dynamicznego rozwoju kolejnych modeli językowych opartych na mechanizmie uwagi. W rezultacie reprezentacje tego samego słowa w różnych znaczeniach ulegały stopniowemu zwiększeniu rozróżnialności. Przykładowo słowo „zamek” użyte w znaczeniu „zamek błyskawiczny” otrzymywało reprezentację wektorową istotnie różniącą się od reprezentacji tego samego słowa w znaczeniu „zamek do drzwi” lub „zamek jako budowla”.

Kolejnym modelem, który stanowił istotny przełom w rozwoju modeli językowych, wykorzystując wielowarstwowy enkoder architektury Transformer do generowania kontekstowych reprezentacji tokenów był model BERT (ang. *Bidirectional Encoder Representations from Transformers*, BERT). W odróżnieniu od klasycznych modeli językowych, BERT nie przewiduje kolejnych słów w sekwencji, lecz wykorzystuje zadanie maskowanego modelowania języka (ang. *Masked Language Modeling*, MLM). W ramach tego zadania losowo wybrany odsetek tokenów w sekwencji wejściowej zostaje zastąpiony specjalnym znacznikiem [MASK], a model uczy się odtwarzać ich oryginalne wartości na podstawie kontekstu obustronnego [6, 13, 36]. Celem MLM jest minimali-

zacja logarytmu prawdopodobieństwa

$$-\sum_{i \in M} \log p_{\Theta}(t_i \mid X_{\setminus M}),$$

gdzie $X = (t_1, \dots, t_N)$ to sekwencja tokenów wejściowych, $M \subseteq \{1, \dots, N\}$ jest zbiorem pozycji zamaskowanych, t_i oznacza prawdziwy token na pozycji i , $X_{\setminus M}$ to sekwencja wejściowa z zamaskowanymi tokenami, $p_{\Theta}(t_i \mid X_{\setminus M})$ reprezentuje prawdopodobieństwo odtworzenia tokenu t_i na podstawie kontekstu wyznaczone przez parametry Θ , zaś Θ to parametry modelu, czyli macierze wag warstw Transformer. Dla każdego tokenu t_i definiowane jest osadzenie wejściowe

$$h_i^{(0)} = E_{\text{tok}}(t_i) + E_{\text{pos}}(i) + E_{\text{seg}}(s_i),$$

gdzie $E_{\text{tok}}(t_i)$ oznacza wektor tokenu t_i , $E_{\text{pos}}(i)$ to wektor pozycyjny dla pozycji i , $E_{\text{seg}}(s_i)$ reprezentuje wektor segmentu wskazujący czy token należy do zdania A czy B , zaś $h_i^{(0)}$ to końcowy wektor wejściowy przekazywany do pierwszej warstwy Transformer. Następnym etapem jest przejście przez L identycznych warstw enkodera, z których każda składa się z bloku wielogłowej samoatencji (ang. *Multi-head self-attention*) oraz sieci przetwarzającej każdą pozycję niezależnie (ang. *Feed-Forward Network*, FFN). Po przejściu przez wszystkie warstwy końcowa reprezentacja tokenu t_i ma postać wektora

$$h_i^{(L)} \in \mathbb{R}^d,$$

gdzie d oznacza wymiar ukryty Transformer. Wektor $h_i^{(L)}$ odzwierciedla znaczenie tokenu w pełnym kontekście zdania. W przeciwieństwie do klasycznych modeli językowych, które operują sekwencyjnie (w przód lub w tył), BERT estymuje prawdopodobieństwa słów w pełnym kontekście dwustronnym, każdy wektor $h_i^{(L)}$ może korzystać z informacji zarówno sprzed, jak i za daną pozycją i . W oryginalnym modelu BERT stosowano dodatkowe zadanie (ang. *Next Sentence Prediction*, NSP), które polegało to na klasyfikacji par zdań (A, B) . Model ten przewidywał czy zdanie B następuje po zdaniu A . Funkcja straty wyrażona była jako suma strat z MLM i NSP

$$\mathcal{L}_{\text{BERT}} = \mathcal{L}_{\text{MLM}} + \mathcal{L}_{\text{NSP}},$$

gdzie $\mathcal{L}_{\text{MLM}}$ to strata z zadania maskowanego modelowania języka, zaś $\mathcal{L}_{\text{NSP}}$ oznacza stratę binarnej klasyfikacji kolejności zdań [6].

Kolejnym modelem w ewolucji modeli opartych na architekturze Transformer do reprezentacji tekstu kontekstowego jest RoBERTa (ang. *Robusted Optimized BERT*, RoBerta), czyli zoptymalizowana wersja BERT-a. Główne zmiany obejmują rezygnację z zadania NSP, zastosowanie dynamicznego maskowania polegającego na zmianie

masek pomiędzy epokami, a także trenowanie na znacznie większych zbiorach danych i z większymi rozmiarami partii uczących. W konsekwencji, funkcja straty w modelu RoBERTa sprowadza się wyłącznie do zadania MLM postaci

$$\mathcal{L}_{\text{RoBERTa}} = \mathcal{L}_{\text{MLM}},$$

gdzie $\mathcal{L}_{\text{MLM}}$ definiowana jest identycznie jak w modelu BERT. Architektura modelu pozostała taka sama, a poprawa wyników wynikała głównie z procedury treningowej [17]. Model BERT zrewolucjonizował przetwarzanie języka naturalnego dzięki pełnemu, dwukierunkowemu uwzględnianiu kontekstu, natomiast RoBERTa pokazała, że można uzyskać jeszcze lepsze wyniki dzięki optymalizacji procesu treningu oraz zastosowaniu większych zbiorów danych. Te dwa modele stanowią podstawę dla kolejnych, bardziej zaawansowanych architektur transformerowych.

Odmiernym podejściem w rozwoju architektur opartych na Transformerze jest model GPT (ang. *Generative Pre-trained Transformer*), który bazuje wyłącznie na strukturze dekodera Transformera i jest trenowany w sposób autoregresyjny. Celem jest modelowanie rozkładu prawdopodobieństwa sekwencji tokenów

$$X = (t_1, t_2, \dots, t_N),$$

gdzie t_N oznacza token na pozycji N . Autoregresyjne modelowanie języka polega na estymacji rozkładu prawdopodobieństwa sekwencji słów poprzez przewidywanie kolejnego elementu na podstawie wszystkich poprzednich tokenów w sekwencji. W oryginalnej pracy [28] cel pretrenowania definiuje się jako maksymalizację log-wiarygodności

$$L_{\text{GPT}}(U) = \sum_{i=1}^N \log p(t_i \mid t_{i-k}, \dots, t_{i-1}; \theta),$$

gdzie U to sekwencja treningowa, k jest rozmiarem okna kontekstu, $p(t_i \mid t_{i-k}, \dots, t_{i-1})$ to warunkowe prawdopodobieństwo wystąpienia tokenu t_i pod warunkiem, że poprzednie tokeny sekwencji to $t_{i-k}, \dots, t_{i-1}$, zaś θ to parametry modelu. Każdy token t_i jest przekształcany w wektor wejściowy z dodanym wektorem pozycyjnym w następujący sposób

$$h_i^{(0)} = E_{\text{tok}}(t_i) + E_{\text{pos}}(i),$$

gdzie $E_{\text{tok}}(t_i)$ oznacza wektor osadzeń słów tokenu t_i , zaś $E_{\text{pos}}(i)$ to osadzenie pozycyjne, umożliwiające modelowi rozróżnianie pozycji tokenów w sekwencji. W celu zachowania autoregresyjnej własności modelu GPT w warstwach dekodera stosuje się maskę trójkątną, która umożliwia przepływ informacji wyłącznie z lewego kontekstu. Maskowana samo-uwaga (ang. *Masked Self-Attention*) jest mechanizmem stosowanym w każdej warstwie dekodera. W jej ramach każdy token może odwoływać się jedynie

do siebie oraz do wszystkich wcześniejszych pozycji w sekwencji, natomiast połączenia z pozycjami przyszłymi są blokowane poprzez zamaskowanie odpowiednich elementów wejścia funkcji softmax (ustawienie ich na wartości silnie ujemne) [13, 32]. Wektory zapytania, kluczy i wartości dla danego tokenu wyznacza się na podstawie reprezentacji z poprzedniej warstwy dekodera, a następnie mechanizm wielogłowej samo-uwagi oblicza ważoną kombinację tych wartości, uwzględniając wyłącznie dozwolone (niezamaskowane) pozycje. Dzięki temu każde kolejne słowo generowane przez model korzysta jedynie z kontekstu historycznego. Następnie uzyskane reprezentacje są przetwarzane przez pozycjo-niezależną sieć typu *feed-forward*, po czym stosowane są normalizacja warstwowa oraz połączenia rezydualne. Cały ten schemat obliczeń jest następnie powtarzany w kolejnych warstwach dekodera, tworząc stos L identycznych warstw [13, 28, 32].

Po treningu model może generować tekst sekwencyjnie, na podstawie kontekstu $(t_1, \dots, t_{i-1})$ obliczany jest rozkład prawdopodobieństwa $p(t_i \mid t_1, \dots, t_{i-1}; \theta)$ i próbkowany jest kolejny token. Proces ten powtarza się iteracyjnie aż do wygenerowania symbolu końca sekwencji lub osiągnięcia maksymalnej długości. W przeciwieństwie do modelu BERT, który uwzględnia kontekst obustronny, GPT ogranicza się wyłącznie do kontekstu lewostronnego. GPT wprowadził do przetwarzania języka naturalnego uniwersalny paradygmat dużych, wstępnie wytrenowanych modeli autoregresyjnych opartych na architekturze dekodera Transformer. Ewolucja tej architektury konsekwentnie zwiększała zdolność modeli do uogólniania wiedzy, kontrolowania procesu generacji oraz stosowania ich w coraz szerszym zakresie zastosowań [28].

Podsumowując, rozwój metod kontekstowych stanowi fundamentalną zmianę w przetwarzaniu języka naturalnego, polegającą na przejściu od statycznych osadzeń do reprezentacji dynamicznych, precyzyjnie oddających znaczenie słowa w zależności od jego kontekstu. Wprowadzenie architektury Transformer oraz mechanizmu uwagi stało się kluczowym punktem zwrotnym, eliminującym ograniczenia sekwencyjnego przetwarzania danych i umożliwiającym modelowanie szerszych zależności. Na tej podstawie wyłoniły się dwa główne podejścia. Pierwsze z nich, reprezentowane przez modele BERT i RoBERTa, wykorzystuje stos enkoderów oraz pełny, dwukierunkowy kontekst, co pozwala na tworzenie głębokich reprezentacji analitycznych, niezbędnych w zadaniach klasyfikacyjnych czy ekstrakcji informacji. Drugie podejście, zrealizowane w modelu GPT, opiera się na strukturze dekodera i autoregresyjnym przewidywaniu kolejnych tokenów, co czyni je optymalnym rozwiązaniem w zastosowaniach generatywnych. Obserwowany rozwój wskazuje na specjalizację modeli w zadaniach analizy lub generacji tekstu, przy równoczesnym upraszczaniu metod treningu i zwiększaniu skali danych.

Rozdział 3

Wybrane architektury i algorytmy przetwarzania języka naturalnego

Reprezentacje wektorowe słów stanowią jedynie pierwszy krok w procesie analizy języka naturalnego i same w sobie nie gwarantują jeszcze zrozumienia kontekstu czy intencji autora. W celu efektywnego wykorzystania tych wielowymiarowych struktur, niezbędne jest zastosowanie wyspecjalizowanych architektur oraz algorytmów zdolnych do interpretacji zależności ukrytych w danych liczbowych. Dopiero połączenie precyzyjnego modelu reprezentacji z wydajnymi metodami wnioskowania umożliwia realizację złożonych zadań NLP. Skuteczne funkcjonowanie współczesnych aplikacji użytkowych opiera się na integracji wyspecjalizowanych mechanizmów przetwarzania informacji oraz algorytmów uczenia maszynowego. Fundamentem systemów operujących na danych wielojęzycznych jest tłumaczenie maszynowe. Umożliwia ono automatyzację przekładu tekstu i przełamywanie barier komunikacyjnych w czasie rzeczywistym. Obok translacji, kluczową rolę w procesie analizy danych odgrywają algorytmy klasyfikacji, pozwalające na przypisywanie obiektów do zdefiniowanych kategorii na podstawie ich cech. W niniejszym rozdziale scharakteryzowano wybrane metody predykcyjne, począwszy od klasycznych podejść statystycznych, takich jak regresja logistyczna, poprzez zaawansowane modele zespołowe typu XGBoost, aż po algorytmy inspirowane biologicznymi układami nerwowymi, reprezentowane przez wielowarstwowy perceptron. Zrozumienie specyfiki tych narzędzi jest niezbędne do właściwego zaprojektowania logiki warstwy analitycznej aplikacji.

3.1. Tłumaczenie maszynowe

Tłumaczenie maszynowe (ang. *Machine Translation*, MT) to proces automatycznego tłumaczenia tekstu lub mowy z jednego języka na drugi za pomocą algorytmów i programów komputerowych. Na przestrzeni lat opracowano różne podejścia do realizacji tego zadania. Początkowo dominowały systemy regułowe wykorzystujące zbiory zasad oraz słowniki lingwistyczne do generowania tłumaczeń. W kolejnych etapach zostały one zastąpione przez systemy statystyczne wspomagane przez modele pro-

babilistyczne. Podejścia te opierały się na analizie dużych korpusów tekstowych oraz automatycznym modelowaniu relacji pomiędzy językiem źródłowym a językiem docelowym. Kluczową rolę odgrywały tu modele leksykalne i frazowe, w których system analizował prawdopodobieństwo wystąpienia odpowiedników pomiędzy segmentami tekstu w obu językach. Ostatecznie, w podejściu statystycznym tłumaczenie realizowano poprzez maksymalizację funkcji prawdopodobieństwa, prowadzącą do wyboru najbardziej prawdopodobnej sekwencji wyrazów w języku docelowym względem sekwencji źródłowej. Główną zaletą tego podejścia była łatwa adaptacja do nowych par językowych oraz zadowalająca skalowalność w porównaniu z wcześniejszymi systemami regułowymi. Z drugiej strony, jakość tłumaczeń wyraźnie spadała w przypadku dłuższych i bardziej złożonych zdań, a także zależała od ilości danych treningowych. Dodatkowo, statystyczne modele tłumaczeniowe nie były w stanie w pełni uwzględniać kontekstu językowego, co prowadziło do utraty spójności semantycznej przede wszystkim w dłuższych fragmentach. W odpowiedzi na wspomniane ograniczenia oraz postęp technologiczny opracowano neuronowe systemy tłumaczenia maszynowego (ang. *Neural Machine Translation*, NMT) [2]. W 2014 roku zaprezentowano model sequence-to-sequence oparty na architekturze enkoder-dekoder z wykorzystaniem dwóch sieci LSTM, który umożliwił tłumaczenie całych zdań z języka angielskiego na francuski [30]. W tym samym okresie zaproponowano również mechanizm uwagi, który umożliwił dekodowaniu dynamiczne koncentrowanie się na najbardziej istotnych fragmentach sekwencji źródłowej podczas generowania tłumaczenia [2, 13].

Klasyczne sieci rekurencyjne (ang. *Recurrent Neural Networks*, RNN) to modele neuronowe przeznaczone do przetwarzania danych sekwencyjnych, w których na każdym kroku czasowym utrzymywana jest ukryta reprezentacja stanu h_t . Reprezentacja ta zależy od poprzedniego stanu ukrytego oraz bieżącego wejścia. Formalnie stan sieci klasycznej RNN obliczany jest za pomocą następującego wzoru

$$h_t = \sigma(W_{xh}x_t + W_{hh}h_{t-1} + b_h),$$

gdzie x_t to wejście w czasie t , h_{t-1} jest poprzednim stanem ukrytym, W_{xh} i W_{hh} to macierze wag, b_h to wektor biasu, zaś σ to nieliniowa funkcja aktywacji. W praktyce w klasycznych sieciach rekurencyjnych najczęściej stosowaną funkcją aktywacji jest funkcja tangensa hiperbolicznego

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$

Natomiast funkcja sigmoidalna postaci

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

wykorzystywana jest głównie w bardziej zaawansowanych architekturach rekurencyjnych, takich jak LSTM i GRU [13]. Funkcja aktywacji wprowadza nieliniowość do modelu, co umożliwia sieciom RNN efektywne modelowanie i uczenie złożonych zależności sekwencyjnych. W klasycznym tłumaczeniu maszynowym zwykle stosuje się architekturę enkoder-dekoder opartą na RNN, w której enkoder przetwarza zdanie źródłowe do postaci wektorowej, zaś dekoder generuje sekwencję tokenów w języku docelowym, obliczając rozkład prawdopodobieństwa kolejnych tokenów krok po kroku ze wzoru

$$P(y|x) = \prod_{t=1}^T P(y_t|y_{<t}, x),$$

gdzie x reprezentuje sekwencję wejściową, y to sekwencja wyjściowa o długości T , y_t to generowany w chwili t token, zaś $y_{<t}$ to dotychczas wygenerowana sekwencja. Przedstawione podejście autoregresywne polega na tym, że kolejny wyraz tłumaczenia jest generowany na podstawie dotychczas przetłumaczonych wyrazów oraz wektorowej reprezentacji całego zdania źródłowego. Klasyczne RNN mają jednak trudności z uczeniem się długoterminowych zależności, głównie ze względu na problem zanikania lub eksplozji gradientów podczas propagacji wstecznej.

W odpowiedzi na te ograniczenia zaproponowano architekturę LSTM rozszerzającą klasyczną sieć rekurencyjną o mechanizmy bramek, które kontrolują przepływ informacji oraz umożliwiające efektywne przechowywanie zależności długoterminowych. Działanie komórki LSTM opisują następujące zależności:

$$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i), \quad (3.1)$$

$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f), \quad (3.2)$$

$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o), \quad (3.3)$$

$$\tilde{c}_t = \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c), \quad (3.4)$$

$$c_t = c_{t-1} + i_t \odot \tilde{c}_t, \quad (3.5)$$

$$h_t = o_t \odot \tanh(c_t), \quad (3.6)$$

gdzie $\odot$ oznacza iloczyn Hadamarda, c_t oznacza stan komórki w czasie t , a funkcje sigmoidalna (σ) i tangens hiperboliczny ($\tanh$) kontrolują przepływ informacji. W porównaniu do standardowych RNN, LSTM wprowadza dwie kluczowe zmiany.

Po pierwsze, jak pokazano w równaniu (3.2), wprowadzono bramkę zapominania f_t . Po drugie, mechanizm ten kontroluje przekazywanie poprzedniego stanu komórki c_{t-1} do aktualnego stanu c_t zgodnie z równaniem (3.5). Bramka zapominania umożliwia precyzyjną kontrolę przepływu informacji, pozwalając sieci zachowywać istotne

dane tak długo, dopóki są potrzebne oraz usuwać je, gdy przestają być istotne. Jeżeli jednak wartość f_t utrzymuje się blisko zera, model będzie systematycznie tracił zapamiętane informacje, co ogranicza jego zdolność do modelowania długoterminowych zależności. Z tego względu bias bramki zapominania b_f jest zazwyczaj inicjalizowany dodatnią wartością (np. 1) na początku treningu, co umożliwia stopniowe uczenie się mechanizmu zapominania [23]. W odpowiedzi na złożoność architektury LSTM proponowano następnie uproszczony wariant, określany jako GRU, który redukuje liczbę bramek oraz eliminuje jawny stan komórki, zachowując zdolność do modelowania zależności długoterminowych. Dzięki mniejszej liczbie parametrów GRU charakteryzuje się niższym kosztem obliczeniowym i często osiąga wyniki porównywalne z LSTM.

Wprowadzenie architektury Transformer w 2017 roku stanowiło istotny przełom w rozwoju neuronowego tłumaczenia maszynowego, umożliwiając skuteczniejsze modelowanie zależności w sekwencjach, zwłaszcza w przypadku długich tekstów, a także eliminując ograniczenia charakterystyczne dla rekurencyjnego przetwarzania danych. Była to pierwsza architektura bazująca wyłącznie na mechanizmach uwagi, bez wykorzystania sieci rekurencyjnych ani splotowych. Autorzy wykazali, że Transformer przewyższa wcześniejsze modele rekurencyjne w zadaniach tłumaczenia przy znacznie krótszym czasie trenowania. Transformatery szybko stały się standardem w tłumaczeniu maszynowym i obecnie są najczęściej wykorzystywanym rozwiązaniem w systemach tłumaczeniowych. Dzięki równoległemu przetwarzaniu sekwencji mogą efektywnie skalować się na duże zbiory danych oraz generować tłumaczenia z jakością zbliżoną do pracy ludzkiej [2, 32].

Architektury transformerowe stosowane w zadaniach tłumaczenia maszynowego składają się z nałożonych warstw enkodera i dekodera. W enkoderze przetwarzane jest zdanie źródłowe, a w dekodерze generowane jest tłumaczenie. W modelach opartych na architekturze Transformer wykorzystywany jest wielogłowy mechanizm uwagi, w którym zapytania, klucze i wartości są najpierw liniowo projektowane do wielu podprzestrzeni, gdzie uwaga jest obliczana niezależnie, a następnie uzyskane wyniki są łączone w jedną reprezentację. Zastosowanie wielogłowej uwagi umożliwia modelowi równoczesne uwzględnianie różnych typów relacji między wyrazami w zdaniu, zwiększa szybkość działania oraz efektywność całego systemu [4]. Każda warstwa Transformera zawiera ponadto warstwę sieci feed-forward stosowaną niezależnie do każdej pozycji sekwencji. Na wejście do modelu dodawane są także wektory pozycyjnego kodowania, które umożliwiają zachowanie informacji o kolejności słów, ponieważ sam mechanizm uwagi nie rozróżnia kolejności tokenów. Cała architektura może być postrzegana jako znormalizowany automatyczny koder sekwencji, złożony z bloków uwagi i warstw FFN. W dekodерze Transformera wykorzystywane są dwa rodzaje mechanizmów uwagi:

- i) maskowana samo-uwaga (ang. masked self-attention), w której zastosowanie maskowania uniemożliwia modelowi dostęp do przyszłych tokenów podczas generowania tłumaczenia,
- ii) uwaga enkoder-dekoder (ang. encoder-decoder attention), umożliwiająca powiązanie reprezentacji generowanych w dekodерze z reprezentacjami zdania źródłowego pochodzącymi z enkodera.

Przedstawiony układ pozwala każdemu tokenowi w dekodерze uwzględniać zarówno kontekst całego zdania źródłowego, jak i dotychczas wygenerowaną część tłumaczenia, co jest kluczowe dla uzyskania wysokiej jakości wyników tłumaczenia [13, 32].

Jednym z pierwszych wielojęzycznych modeli opartych na architekturze Transformer zaprojektowanych do zadań tłumaczenia maszynowego, był mBART, wstępnie wytrenowany na dużych korpusach tekstowych obejmujących wiele języków. Proces wstępnego uczenia tego modelu opierał się na rekonstrukcji celowo uszkodzonego tekstu, co pozwoliło na efektywne modelowanie zarówno tekstu źródłowego, jak i docelowego. Dzięki temu mBART umożliwia tłumaczenie pomiędzy wieloma językami bez konieczności specjalistycznego dostrajania, choć dodatkowe dostrajanie do konkretnych par językowych lub dziedzin tematycznych w większości przypadków istotnie poprawia jakość tłumaczeń. Jedną z najczęściej stosowanych wersji modelu jest mBART-50, który obsługuje 50 języków i jest szeroko wykorzystywany w wielojęzycznych systemach tłumaczeniowych [18].

Wśród kolejnych istotnych modeli należy wymienić M2M-100, zaawansowany system opracowany przez Meta AI, umożliwiający bezpośrednie tłumaczenie pomiędzy 100 językami bez konieczności korzystania z języka pośredniego. Takie podejście znacząco poprawia jakość tłumaczeń, szczególnie w parach językowych o niskiej dostępności danych. Model ten został wytrenowany na dużych, wielojęzycznych zbiorach i obsługuje łącznie aż 9 900 kierunków tłumaczeń. M2M-100 jest szeroko stosowany w systemach tłumaczeniowych wymagających zarówno rozległego pokrycia językowego, jak i wysokiej jakości tłumaczeń [8].

Współczesne klasyczne modele transformerowe przeznaczone wyłącznie do tłumaczenia maszynowego opierają się na tej samej architekturze enkoder-dekoder. Różnice pomiędzy poszczególnymi rozwiązaniami dotyczą przede wszystkim zakresu i jakości danych treningowych, a także parametrów architektury, takich jak liczba warstw, rozmiar reprezentacji czy liczba głów mechanizmu uwagi. Do powszechnie stosowanych rozwiązań należą architektury takie jak Marian NMT (MarianMT), NLLB-200, T5 wraz z jego wielojęzycznym wariantem mT5 oraz ByT5, które są szeroko wykorzystywane w systemach tłumaczeniowych o charakterze komercyjnym. Budowanie modeli

tej klasy od zera wymagałoby bardzo dużych zasobów obliczeniowych oraz obszer-nych zbiorów danych, co w praktyce uniemożliwia osiągnięcie jakości porównywalnej z modelami tworzonymi przez duże organizacje badawcze i przemysłowe. Najczęściej wykorzystuje się gotowe, duże modele, które są następnie dostosowywane do konkretnych zastosowań (ang. fine-tuning). Do tłumaczenia maszynowego mogą być również wykorzystywane architektury wyłącznie dekodujące, takie jak modele z rodziny GPT. Modele tego typu osiągają zadowalające wyniki w zadaniach tłumaczeniowych, a jednocześnie znajdują szerokie zastosowanie w wielu innych zadaniach przetwarzania języka naturalnego. Należy jednak podkreślić, że charakteryzują się one bardzo dużą liczbą parametrów, co sprawia, że ich lokalne wdrażanie jest często niepraktyczne. W konsekwencji, ich wykorzystanie odbywa się zazwyczaj poprzez interfejsy programistyczne aplikacji (ang. Application Programming Interface, API), co wiąże się z dodatkowymi kosztami eksploatacyjnymi.

Tłumaczenie maszynowe mimo znaczącego postępu, wciąż mierzy się z wyzwaniami, takimi jak brak wystarczających danych dla rzadszych języków oraz trudności w adaptacji do specyficznych domen tematycznych. Modele często popełniają błędy przy tłumaczeniu idiomów i struktur zależnych od szerokiego kontekstu, co wymusza konieczność stosowania końcowej edycji wyników przez człowieka. Nowoczesne architektury, takie jak Transformer, choć skuteczne, wymagają znacznej mocy obliczeniowej i są wrażliwe na niską jakość danych treningowych. Pomimo tych trudności, istotną zaletą podejścia neuronowego jest automatyczne generowanie wektorowych reprezentacji słów, które precyzyjnie oddają ich semantykę. Uzyskane w ten sposób osadzenia umożliwiają transfer wiedzy i znajdują zastosowanie w innych zadaniach NLP, takich jak klasyfikacja tekstu, co stanowi jeden z głównych kierunków dalszego rozwoju tej technologii. W przeciwieństwie do NMT, tradycyjne modele uczenia maszynowego nie sprawdzają się w tłumaczeniu, gdyż nie są w stanie efektywnie generować sekwencji wyrazów. Ich zastosowanie ogranicza się głównie do klasyfikacji, gdzie kluczową rolę odgrywa jakość statycznych reprezentacji wektorowych tekstu. Przyszłość dziedziny wiąże się z rozwojem wielozadaniowych modeli generatywnych oraz doskonaleniem metod adaptacji do konkretnych dziedzin. Ostatecznie dąży się do stworzenia systemów bardziej uniwersalnych, które dzięki lepszemu rozumieniu kontekstu wyeliminują obecne ograniczenia technologiczne i językowe [32, 34].

3.2. Algorytmy uczenia maszynowego w klasyfikacji tekstu

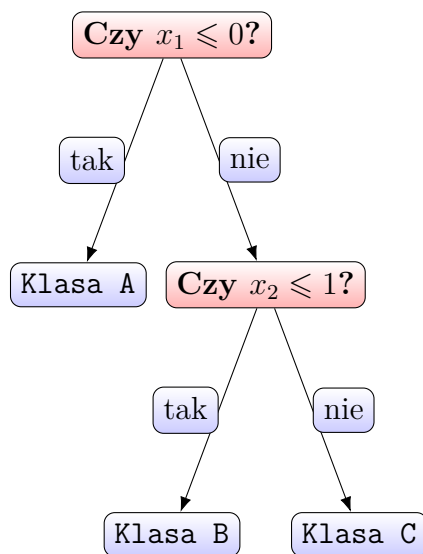
W przetwarzaniu języka naturalnego osadzenia słów to reprezentacje tekstu w postaci wektorów liczbowych, które umożliwiają komputerom efektywne operowanie na

danych tekstowych. Na takich reprezentacjach opierają się tradycyjne algorytmy uczenia maszynowego, znajdujące szerokie zastosowanie przede wszystkim w zadaniach klasyfikacji tekstu, w których celem jest przypisanie tekstom jednej lub wielu etykiet. Modelowanie wektorów osadzeń stanowi istotne wyzwanie, przede wszystkim ze względu na ich wysoką wymiarowość, która prowadzi do wzrostu złożoności oraz kosztów obliczeniowych. Zjawisko to, określane mianem klątwy wymiarowości, oznacza, że wykładniczy wzrost liczby wymiarów przestrzeni wymaga bardzo dużej liczby przykładów treningowych, a klasyczne miary odległości stopniowo tracą swoje intuicyjne znaczenie [1, 34]. Ponadto, w statystycznych metodach osadzania słów rzadkość danych w przestrzeni cech stanowi dodatkowe ograniczenie utrudniające efektywne rozdzielanie klas. Niezależnie od doboru reprezentacji, wyzwaniem badawczym pozostaje również selekcja odpowiednich modeli predykcyjnych, które umożliwią efektywne wykorzystanie dostępnej informacji w danych. W pracy przedstawione zostały trzy wybrane grupy modeli predykcyjnych, które stanowią podstawę dalszych analiz: modele liniowe, modele drzewiaste oraz sztuczne sieci neuronowe.

Najprostszym liniowym podejściem do klasyfikacji tekstu z wykorzystaniem osadzeń słów jest regresja logistyczna. W przypadku binarnej regresji logistycznej model estymuje prawdopodobieństwo przynależności danej próbki do klasy pozytywnej (np. pozytywnego sentymentu). Dla problemów wieloklasowych stosuje się jej uogólnienie w postaci funkcji softmax, która zwraca rozkład prawdopodobieństwa przynależności do poszczególnych klas [1]. Regresja logistyczna charakteryzuje się niską złożonością obliczeniową, prostotą implementacji, dobrą odpornością na przeuczenie oraz skutecznością w pracy z rzadkimi reprezentacjami danych. Jednocześnie do jej głównych ograniczeń należy brak zdolności do modelowania złożonych, nieliniowych zależności oraz interakcji pomiędzy cechami. Szczegółowy opis modelu oraz jego własności matematycznych można znaleźć w następujących pozycjach [1, 13].

Drzewa decyzyjne (ang. *Decision Trees*) są nadzorowanymi metodami uczenia maszynowego stosowanymi w zadaniach klasyfikacji oraz regresji. Model drzewa dokonuje hierarchicznego podziału przestrzeni cech na rozłączne podobszary decyzyjne, wykorzystując sekwencję reguł opartych na warunkach logicznych. Jednym z najczęściej stosowanych algorytmów konstrukcji drzew jest CART (ang. *Classification and Regression Trees*) [19]. Algorytm ten buduje drzewo poprzez rekurencyjny podział danych na coraz mniejsze podzbiory według wartości atrybutów. W każdym kroku wybierana jest taka reguła, która dzieli dane na dwa podzbiory (lewy i prawy) w sposób maksymalizujący jakość podziału, mierzoną redukcją nieczystości węzłów. Proces ten trwa do momentu spełnienia kryteriów stopu, takich jak: osiągnięcie maksymalnej głębokości drzewa, minimalnej liczby próbek w węźle lub braku dalszej poprawy jako-

ści podziału. W strukturze drzewa węzły wewnętrzne odpowiadają testom na cechach, natomiast liście zawierają końcowe predykcje. W zadaniach klasyfikacyjnych predykcja w liściu odpowiada klasie dominującej (ang. majority vote), natomiast w regresji jest to średnia wartość zmiennej celu w danym liściu [19, 34]. Rysunek 3.1 przedstawia uproszczone drzewo decyzyjne, w którym węzeł główny testuje warunek dotyczący cechy x_1 . Dla gałęzi odpowiadającej wartości „tak” obserwacja trafia do liścia przypisanego do klasy A, natomiast w przypadku wartości „nie” wykonywany jest kolejny test na cechę x_2 . Liście końcowe reprezentują wynikowe klasy przypisane na podstawie spełnionych warunków.



Rysunek 3.1: Schemat przykładowego drzewa decyzyjnego

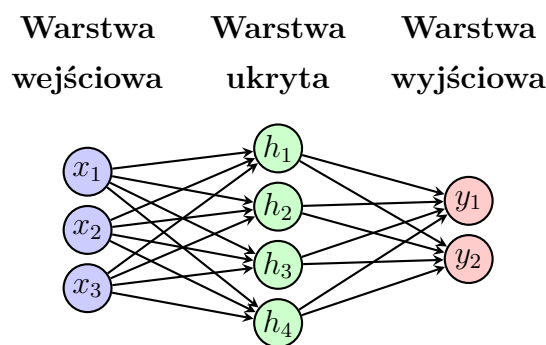
Źródło: Opracowanie własne

Pojedyncze drzewo decyzyjne jest prostym i interpretowalnym klasyfikatorem, jednak cechuje się skłonnością do nadmiernego dopasowania (ang. *overfitting*) oraz relatywnie wysoką wariancją. Z tego względu w praktycznych zastosowaniach powszechnie wykorzystuje się metody zespołowe, polegające na agregacji predykcji wielu modeli (np. drzew decyzyjnych) w celu zwiększenia stabilności oraz poprawy jakości rozwiązania. Do najczęściej stosowanych technik tego rodzaju należą bagging oraz boosting.

XGBoost (ang. *eXtreme Gradient Boosting*) jest wydajną i skalowalną implementacją metody gradientowego boostingu, która jest przystosowana do wykorzystywania drzew decyzyjnych jako modeli bazowych. W zastosowaniach klasyfikacyjnych model działa poprzez sumowanie surowych predykcji generowanych przez kolejne drzewa, które są uczone sekwencyjnie w celu korygowania błędów popełnianych przez wcześniejsze iteracje. Następnie po zsumowaniu predykcji wszystkich drzew końcowy wynik jest przekształcany w prawdopodobieństwa przynależności do klas za pomocą funkcji

transformującej wynik modelu na rozkład prawdopodobieństwa, takiej jak funkcja sigmoidalna lub softmax. XGBoost rozszerza klasyczny gradient boosting o mechanizmy kontroli złożoności drzew, efektywne algorytmy optymalizacji oraz techniki przyspieszające obliczenia, dzięki czemu osiąga wysoką skuteczność predykcyjną przy zachowaniu dobrej skalowalności. W analizie danych tabelarycznych metody drzewiaste należą do jednych z najsukuteczniejszych podejść predykcyjnych, co znajduje potwierdzenie w licznych artykułach naukowych [5, 9].

Sieci neuronowe (ang. *Neural Networks*, NN) to modele matematyczne inspirowane budową biologicznych sieci neuronalnych. Klasyczny perceptron wielowarstwowy (ang. *multilayer perceptron*, MLP) składa się z warstwy wejściowej, co najmniej jednej warstwy ukrytej oraz warstwy wyjściowej i stanowi przykład gęstej sieci neuronowej (ang. *Dense Neural Network*), w której warstwy są w pełni połączone. Oznacza to, że każdy neuron warstwy l jest połączony z każdym neuronem warstwy $l - 1$ (ang. *fully connected*). Liczba neuronów w warstwie wejściowej odpowiada wymiarowi wektora wejściowego modelu, natomiast liczba warstw ukrytych oraz liczba neuronów w tych warstwach stanowią hiperparametry dobierane przez analityka. Struktura warstwy wyjściowej zależy od rodzaju rozwiązywanego zadania. W przypadku klasyfikacji odpowiada ona liczbie kategorii zmiennej zależnej, a wyjścia neuronów są interpretowane jako prawdopodobieństwa przynależności do klas, natomiast w zadaniach regresyjnych najczęściej stosowany jest pojedynczy neuron wyjściowy określający prognozowaną wartość. Każdy neuron otrzymuje sygnały z poprzedniej warstwy, oblicza ich ważoną sumę i przetwarza ją za pomocą nieliniowej funkcji aktywacji [16, 34]. Na Rysunku 3.2 przedstawiono przykładowy schemat sieci MLP z jedną warstwą ukrytą.



Rysunek 3.2: Schemat sieci MLP z jedną warstwą ukrytą

Źródło: Opracowanie własne

Dzięki zastosowaniu wielu warstw ukrytych oraz nieliniowych funkcji aktywacji sieci neuronowe są zdolne do modelowania złożonych, nieliniowych zależności pomiędzy danymi wejściowymi a wyjściem. Z tego względu sieci neuronowe stanowią obecnie

jedną z kluczowych klas modeli wykorzystywanych w nowoczesnych metodach uczenia maszynowego [16, 34].

Zaprezentowane algorytmy reprezentują trzy odmienne podejścia do modelowania, z których każde charakteryzuje się innym kompromisem między interpretowalnością, a zdolnością do odwzorowania złożonych funkcji. Modele liniowe, takie jak regresja logistyczna, oferują transparentność działania i szybkość uczenia, lecz ich skuteczność jest ograniczona w przypadku występowania silnych zależności nieliniowych. Ograniczenie to jest skutecznie niwelowane przez metody zespołowe (np. XGBoost), które osiągają wysoką precyzję dzięki agregacji wielu prostych modeli drzewiastych oraz przez sztuczne sieci neuronowe (MLP), które zdolność do modelowania nieliniowości zawdzięczają rozbudowanej architekturze i ogromnej liczbie parametrów. W praktyce rzadko możliwe jest wskazanie z góry najlepszego rozwiązania, dlatego kluczowym etapem prac jest faza eksperymentalna, pozwalająca na empiryczną weryfikację skuteczności poszczególnych algorytmów na konkretnym zbiorze danych.

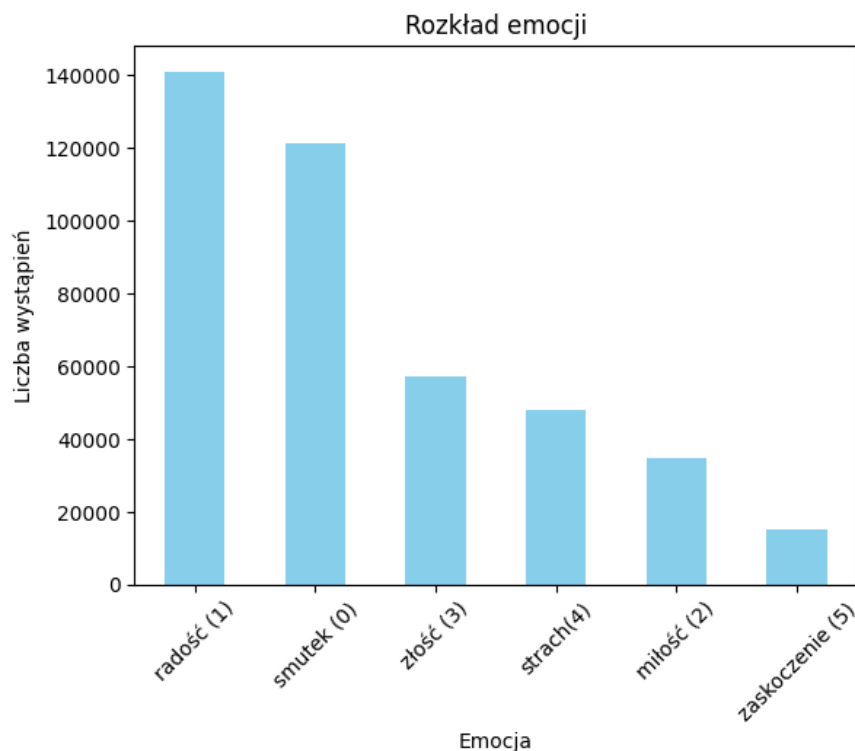
Rozdział 4

Analiza wyników i implementacja aplikacji

Niniejszy rozdział poświęcony jest części praktycznej pracy obejmującej ocenę skuteczności wybranych metod osadzania słów w zadaniu klasyfikacyjnym oraz implementację aplikacji realizującej tłumaczenie maszynowe i prognozowanie emocji w tekście. Głównym celem pracy było porównanie wybranych metod reprezentacji tekstu zawierających podejścia statystyczne oraz statyczne i kontekstowe metody osadzania słów. Równolegle przeprowadzona została weryfikacja jakości etykietowania danych, obejmująca identyfikację oraz automatyczną korektę błędnych oznaczeń z wykorzystaniem autorskiego systemu opartego na modelu GPT-4o-mini. Kończącym etapem pracy było opracowanie aplikacji umożliwiającej wielojęzyczne tłumaczenie tekstu wprowadzanego przez użytkownika oraz jego klasyfikację do jednej z sześciu kategorii emocji, wraz z wyświetleniem prawdopodobieństwa przynależności do poszczególnych klas.

4.1. Zadanie klasyfikacji tekstu

W części praktycznej wykorzystano zbiór danych Twitter Emotion Dataset [7], dostępny publicznie na platformie Kaggle. Analizowane dane pochodzą z platformy X (dawniej Twitter) i obejmują krótkie teksty (ang. *tweets*) publikowane przez użytkowników na różne tematy. Zbiór danych liczy około 330 000 obserwacji, z których każda zawiera treść tweeta w języku angielskim oraz przypisaną etykietę jednej z sześciu emocji: radości, smutku, złości, strachu, miłości oraz zaskoczenia. Na Rysunku 4.1 zaprezentowano rozkład klas emocji w analizowanym zbiorze.



Rysunek 4.1: Rozkład liczby obserwacji w poszczególnych klasach emocji

Na podstawie Rysunku 4.1 można zaobserwować istotne niezbalansowanie klas, przy czym najliczniej reprezentowanymi kategoriami są radość (ang. *joy*) oraz smutek (ang. *sadness*). Emocje te występują znacznie częściej niż złość (ang. *anger*) oraz strach (ang. *fear*), natomiast miłość (ang. *love*) i zaskoczenie (ang. *surprise*) stanowią najmniej liczną część analizowanego zbioru danych. Nierównomierna liczba przykładów treningowych w poszczególnych klasach oznacza, że algorytmy mogą wykazywać tendencję do faworyzowania emocji najczęściej występujących. Zjawisko to należy uwzględnić na etapie trenowania i ewaluacji modeli, stosując odpowiednie metryki lub techniki równoważenia danych.

Na etapie wstępnego przetwarzania danych przeprowadzono również analizę jakości zbioru pod kątem występowania niespójności w dokonanych anotacjach. Zidentyfikowano przypadki identycznych treści tekstowych przypisanych do różnych klas emocji, co przedstawiono w Tabeli 4.1.

Tabela 4.1: Przykłady identycznych tekstów przypisanych do różnych klas

text	label
i feel like that combo is kinda weird	5
i feel like that combo is kinda weird	4
i just feel really helpless and heavy hearted	4
i just feel really helpless and heavy hearted	0

Tego rodzaju obserwacje świadczą o problematycznej strukturze zbioru danych, która mogłaby negatywnie wpłynąć na proces uczenia modeli klasyfikacyjnych. W celu zapewnienia jednoznaczności oznaczeń danych tekstowych oraz poprawy jakości zbioru uczącego zastosowano procedurę czyszczenia danych polegającą na usunięciu duplikatów. Zatem dla każdego unikalnego tekstu pozostawiono wyłącznie jeden rekord, niezależnie od liczby wcześniej przypisanych mu etykiet emocji. W rezultacie każdy tweet występuje w zbiorze danych tylko raz i jest przypisany do jednej, najbardziej prawdopodobnej klasy emocji.

Kolejnym etapem przygotowania danych do dalszych analiz był ich podział na trzy rozłączne podzbiory. W tym celu losowe 70% zgromadzonych obserwacji zostało przydzielonych do zbioru treningowego (ang. *train dataset*). Tak duża liczność próby uczącej jest standardowym i zalecanym wyborem w zadaniach przetwarzania języka naturalnego, gdyż pozwala modelowi na skuteczne wychwycenie złożonych zależności semantycznych oraz różnorodnych sposobów wyrażania tych samych emocji przez użytkowników. Następne losowe 15% danych wyodrębniono jako zbiór walidacyjny (ang. *validation dataset*), który służy do monitorowania postępów uczenia, dostrajania hiperparametrów oraz wyboru najlepszej wersji klasyfikatora. Ostatnie 15% rekordów stanowi zbiór testowy (ang. *test dataset*) umożliwiający końcową, obiektywną weryfikację zdolności modelu do poprawnego rozpoznawania emocji w nowych, wcześniej nieznanach tekstach.

4.1.1. Metryki ewaluacyjne modeli

W celu rzetelnej oceny skuteczności modeli uczenia maszynowego niezbędne jest zdefiniowanie odpowiednich miar opartych na macierzy pomyłek (ang. *Confusion Matrix*). Punktem wyjścia do analizy skuteczności klasyfikatora binarnego jest zestawienie predykcji modelu z rzeczywistymi etykietami. W ramach tej analizy wyróżnia się cztery rozłączne scenariusze. W przypadku poprawnych klasyfikacji model identyfikuje obiekt klasy pozytywnej (ang. *True Positive, TP*) lub prawidłowo odrzuca obiekt klasy negatywnej (ang. *True Negative, TN*). W przypadku błędnej klasyfikacji pomyłka

może przyjąć jedną z dwóch form. Jeżeli model fałszywie prognozuje wystąpienie klasy pozytywnej, występuje błąd I rodzaju (ang. *False Positive*, *FP*). Natomiast sytuacja odwrotna, w której model nie wykryje poprawnego wystąpienia klasy pozytywnej, stanowi błąd II rodzaju (ang. *False Negative*, *FN*). Wartości te stanowią podstawę do wyznaczenia wszystkich dalszych wskaźników jakości modeli predykcyjnych [10].

Najbardziej intuicyjną miarą oceny skuteczności modelu jest jego dokładność (ang. *Accuracy*), definiowana jako stosunek wszystkich poprawnych predykcji do ogólnej liczby obserwacji, wyrażona następującym wzorem

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}.$$

Metryka ta, mimo że pozwala na szybką ocenę ogólnej sprawności systemu, posiada istotne ograniczenie. W przypadku zbiorów niezbalansowanych, gdzie jedna klasa znacząco dominuje, wysoka wartość dokładności może być myląca i może nie odzwierciedlać rzeczywistej zdolności modelu do detekcji klasy mniejszościowej.

W celu uzyskania bardziej szczegółowego obrazu działania modelu, zwłaszcza w kontekście błędów I i II rodzaju, stosuje się pary miar: precyzję oraz czułość. Precyzja (ang. *Precision*) określa wiarygodność pozytywnych wskazań modelu, informując jaka część obiektów sklasyfikowanych jako pozytywne faktycznie należy do tej klasy, określona jest wzorem

$$Precision = \frac{TP}{TP + FP}.$$

Z kolei czułość (ang. *Recall*), nazywana również pełnością, mierzy zdolność modelu do wykrywania wszystkich rzeczywistych przypadków klasy pozytywnej. Miara ta jest następującej postaci

$$Recall = \frac{TP}{TP + FN}.$$

W praktyce często występuje relacja wymienna między tymi dwiema miarami, ponieważ optymalizacja modelu pod kątem wysokiej precyzji może skutkować spadkiem czułości i odwrotnie. Rozwiązaniem tego problemu jest zastosowanie miary F1 (ang. *F1-score*), która stanowi średnią harmoniczną precyzji oraz czułości. Miara ta jest definiowana jako

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} = \frac{2TP}{2TP + FP + FN}.$$

Dzięki takiej konstrukcji, miara F1 osiąga wysokie wartości jedynie wtedy, gdy zarówno precyzja, jak i czułość są na odpowiednim poziomie, co czyni ją kluczowym wskaźnikiem w ocenie modeli uczonych na danych niezbalansowanych.

Kolejnym istotnym wskaźnikiem pozwalającym na rzetelną weryfikację modeli dla danych niezbalansowanych jest dokładność zbalansowana (ang. *Balanced Accuracy*). Miara ta eliminuje problem dominacji klasy większościowej, który występuje przy standardowej dokładności, traktując błędy w obu klasach z równą wagą. Dokładność zbalansowana definiowana jest jako średnia arytmetyczna opisana wzorem

$$\text{Balanced Accuracy} = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right).$$

Dzięki uśrednieniu wyników dla obu klas (pozytywnej i negatywnej), dokładność zbalansowana daje bardziej obiektywny obraz sytuacji. W przypadku gdy model dobrze klasyfikuje klasę dominującą, ale ignoruje klasę mniejszościową, standardowa dokładność często przyjmuje wysokie wartości, natomiast poprawność zbalansowana poprawnie ulega obniżeniu.

W kontekście klasyfikacji wieloklasowej lub w sytuacjach, w których istotne jest równorzędne traktowanie każdej klasy, niezależnie od jej liczebności, stosuje się makro-średnią miary F1 (ang. *Macro-averaged F1-score*). Podejście to polega na wyznaczeniu wartości miary F1 niezależnie dla każdej z N klas, a następnie na obliczeniu ich średniej arytmetycznej, która jest wyrażona następującym wzorem

$$F1_{macro} = \frac{1}{N} \sum_{i=1}^N F1_i,$$

gdzie N oznacza całkowitą liczbę klas, zaś $F1_i$ to wartość miary F1 obliczona dla i -tej klasy traktowanej jako klasa pozytywna (podczas gdy wszystkie pozostałe są negatywne). W celu uzyskania wysokiej wartości wskaźnika $F1_{macro}$ model powinien wykazywać wysoką skuteczność dla każdej klasy z osobna. Nawet perfekcyjna klasyfikacja klasy większościowej nie rekompensuje niskiej skuteczności na klasie mniejszościowej, co czyni tę metrykę użytecznym narzędziem w zadaniach, w których kluczowe jest wykrywanie trudnych, rzadziej występujących przypadków [10].

W analizie problemów wieloklasowych, takich jak omawiane zadanie, do wizualizacji wyników wykorzystana została macierz pomyłek. Jest to macierz kwadratowa M o wymiarach $N \times N$, gdzie N oznacza liczbę klas (w tym przypadku $N = 6$). Pojedynczy element macierzy $M_{i,j}$ (i oznacza indeks wiersza, a j kolumny) reprezentuje licznosc próbek, która faktycznie należy do klasy i , lecz została sklasyfikowana przez model do klasy j . W celu poprawnej interpretacji wyników w pracy przyjęto standardową konwencję, w której wiersze macierzy reprezentują rzeczywiste etykiety klas (ang. *True*), zaś kolumny to etykiety przewidywane przez model (ang. *Predicted*). Macierz pomyłek stanowi podstawowe narzędzie analizy błędów w zadaniach klasyfikacyjnych, umożliwiając ocenę zarówno poprawnych, jak i błędnych predykcji dla poszczególnych klas. Pozwala ona na identyfikację najczęściej popełnianych błędów

oraz ocenę relacji pomiędzy klasami. Interpretacja macierzy pomyłek opiera się na dwóch głównych elementach:

1. przekątna główna ($i = j$): zawiera liczby przypadków poprawnie sklasyfikowanych (TP_i), gdzie i to dana klasa z N klas. Na wizualizacji wyższe wartości na przekątnej, reprezentowane intensywniejszym odcieniem koloru, wskazują na większą skuteczność modelu w poprawnej identyfikacji danej kategorii,
2. obszar poza przekątną ($i \neq j$): wartości te obrazują błędy klasyfikacji. Ich analiza pozwala na rozróżnienie dwóch typów pomyłek w kontekście wybranej klasy:
 - i) suma wartości w wierszu i (z wyłączeniem elementu na przekątnej) wyznacza błąd FN_i . Wartość ta wskazuje ile próbek należących do klasy i zostało błędnie przypisanych do innych kategorii (tzw. przeoczenia),
 - ii) suma wartości w kolumnie j (z wyłączeniem elementu na przekątnej) wyznacza błąd FP_j . Wartość ta informuje ile próbek z innych klas zostało błędnie rozpoznanych jako klasa j (tzw. fałszywe alarmy).

Macierz pomyłek umożliwia również identyfikację specyficznych par klas, które są ze sobą mylone przez model, co jest niemożliwe przy analizie wyłącznie ogólnej dokładności oraz innych miar jakości modeli. Schematyczną strukturę analizowanej macierzy pomyłek przedstawiono na Rysunku 4.2.

Klasa Rzeczywista	Klasa 0	TP_0 (Poprawne)	$E_{0,1}$ (Błąd)	$E_{0,2}$ (Błąd)	$E_{0,3}$ (Błąd)	$E_{0,4}$ (Błąd)	$E_{0,5}$ (Błąd)
	Klasa 1	$E_{1,0}$ (Błąd)	TP_1 (Poprawne)	$E_{1,2}$ (Błąd)	$E_{1,3}$ (Błąd)	$E_{1,4}$ (Błąd)	$E_{1,5}$ (Błąd)
	Klasa 2	$E_{2,0}$ (Błąd)	$E_{2,1}$ (Błąd)	TP_2 (Poprawne)	$E_{2,3}$ (Błąd)	$E_{2,4}$ (Błąd)	$E_{2,5}$ (Błąd)
	Klasa 3	$E_{3,0}$ (Błąd)	$E_{3,1}$ (Błąd)	$E_{3,2}$ (Błąd)	TP_3 (Poprawne)	$E_{3,4}$ (Błąd)	$E_{3,5}$ (Błąd)
	Klasa 4	$E_{4,0}$ (Błąd)	$E_{4,1}$ (Błąd)	$E_{4,2}$ (Błąd)	$E_{4,3}$ (Błąd)	TP_4 (Poprawne)	$E_{4,5}$ (Błąd)
	Klasa 5	$E_{5,0}$ (Błąd)	$E_{5,1}$ (Błąd)	$E_{5,2}$ (Błąd)	$E_{5,3}$ (Błąd)	$E_{5,4}$ (Błąd)	TP_5 (Poprawne)
		Klasa 0	Klasa 1	Klasa 2	Klasa 3	Klasa 4	Klasa 5
		Klasa Przewidziana					

Rysunek 4.2: Schemat interpretacji macierzy pomyłek dla problemu klasyfikacji 6-klasowej

W procesie ewaluacji modeli klasyfikacyjnych wykorzystano zestaw komplementarnych metryk pozwalających na rzetelną ocenę skuteczności predykcji w zadaniu rozpoznawania emocji. Podstawowym miernikiem była dokładność, jednak ze względu na specyfikę problemu kluczową rolę odegrały również dokładność zbalansowana oraz makrośrednia miary F1. Zastosowanie tych wskaźników pozwoliło na obiektywne porównanie jakości działania modeli niezależnie od liczebności poszczególnych klas w zbiorze danych. Architektury wszystkich modeli oraz konfiguracja ich parametrów pozostały stałe, co zapewniało porównywalność uzyskanych wyników i rzetelność formułowanych wniosków.

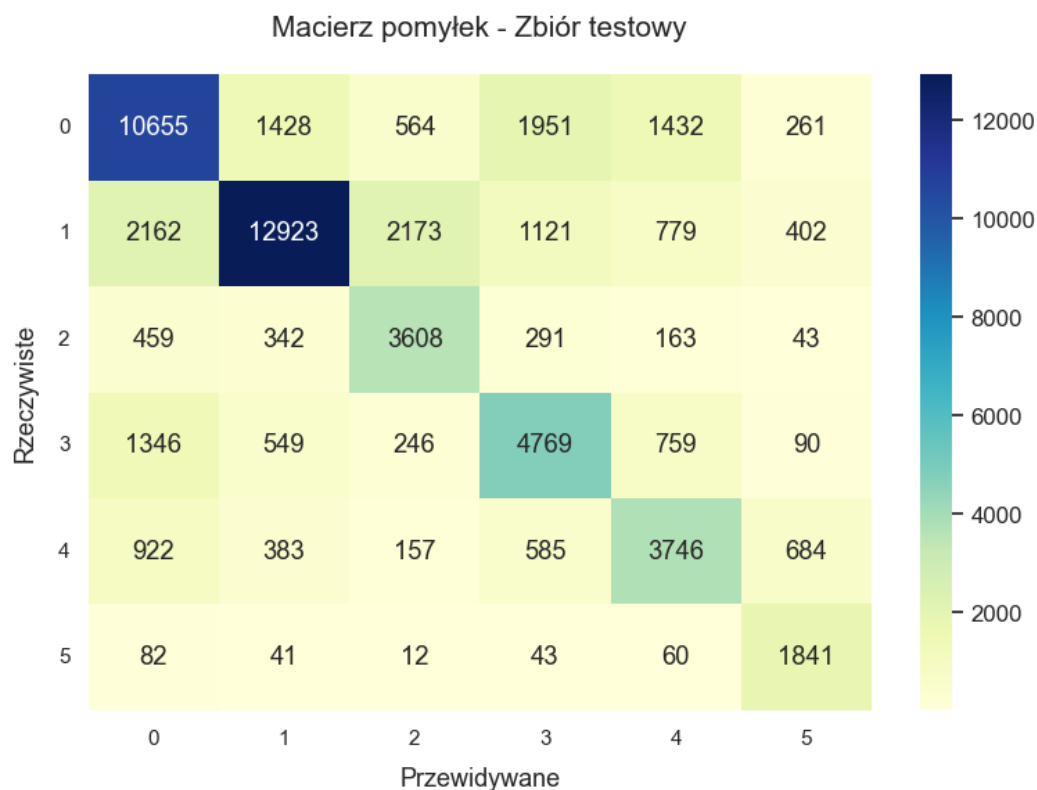
4.1.2. Podejścia statystyczne

Wstępna analiza obejmowała modele oparte na tradycyjnych metodach statystycznych, stanowiące punkt odniesienia dla bardziej złożonych rozwiązań. W ramach tej grupy badań skupiono się przede wszystkim na technice TF-IDF, analizując jak klasyczna reprezentacja oparta na częstości występowania danych słów sprawdza się w ekstrakcji cech niezbędnych do poprawnej identyfikacji emocji w tekście. Do zadania klasyfikacyjnego wykorzystano trzy modele: regresję logistyczną, algorytm XGBoost oraz sieć neuronową MLP. Zestawienie uzyskanych rezultatów dla zbioru testowego prezentuje Tabela 4.2.

Tabela 4.2: Szczegółowe wyniki metryk dla modeli opartych na TF-IDF na zbiorze testowym

Model	Accuracy	Balanced Accuracy	$F1_{macro}$
Logistic Regression	0.6578	0.6881	0.6396
XGBoost	0.6293	0.5790	0.6151
MLP	0.6628	0.6855	0.6452

Analiza metryk przedstawionych w Tabeli 4.2 wskazuje, że różnice w skuteczności pomiędzy poszczególnymi algorytmami są marginalne. Sugeruje to, że głównym ograniczeniem jest sam sposób reprezentacji danych, a nie dobór klasyfikatora. Najlepsze wyniki dla wszystkich miar osiągnął model MLP. W związku z tym dla tego modelu wygenerowano macierz pomyłek w celu dokładniejszej analizy popełnionych błędów oraz identyfikacji klas sprawiających największe trudności w rozpoznawaniu. Na Rysunku 4.3 przedstawione zostały wyniki modelu MLP na zbiorze testowym.



Rysunek 4.3: Macierz pomyłek dla modelu TF-IDF na zbiorze testowym

Analiza macierzy pomyłek pozwoliła na zidentyfikowanie charakterystycznych zależności oraz ograniczeń w procesie klasyfikacji emocji. Obserwując rozkład predykcji, można zauważyć, że model wykazuje tendencję do faworyzowania klas o największej liczebności w zbiorze uczącym, czyli klasy 0 oraz klasy 1. Jest to typowe zjawisko dla metod statystycznych trenowanych na danych niezbalansowanych. Najwyższą skuteczność separacji odnotowano w przypadku klasy 5, dla której model generuje stosunkowo najmniej błędów I i II rodzaju, pomimo że jest to jedna z najmniej licznych klas. Zależność ta sugeruje, że słownictwo charakteryzujące tę emocję jest unikalne, co pozwala algorytmowi TF-IDF na łatwe wyodrębnienie kluczowych cech bez konieczności analizy głębszego kontekstu. Zdecydowanie największym wyzwaniem dla tego modelu okazało się rozróżnienie między klasami 0 i 1, gdzie obserwuje się wysoki poziom wzajemnych pomyłek. Dodatkowo widoczne jest istotne rozmycie predykcji dla klasy 0, która jest często błędnie klasyfikowana jako klasa 3 oraz 4.

Podsumowując, uzyskane wyniki wskazują, że podejście statystyczne, ze względu na swoje założenia, okazało się niewystarczające do skutecznej realizacji zadania. Metoda TF-IDF traktuje tekst jako zbiór nieuporządkowanych tokenów, ignorując relacje sekwencyjne i kontekst semantyczny. Struktura błędów potwierdza, że w przypadku emocji opisywanych zbliżonym zasobem leksykalnym, prosta analiza częstości występo-

wania słów jest niewystarczająca do wychwycenia subtelnych niuansów, co uzasadnia konieczność zbadania bardziej złożonych rozwiązań w dalszej części pracy.

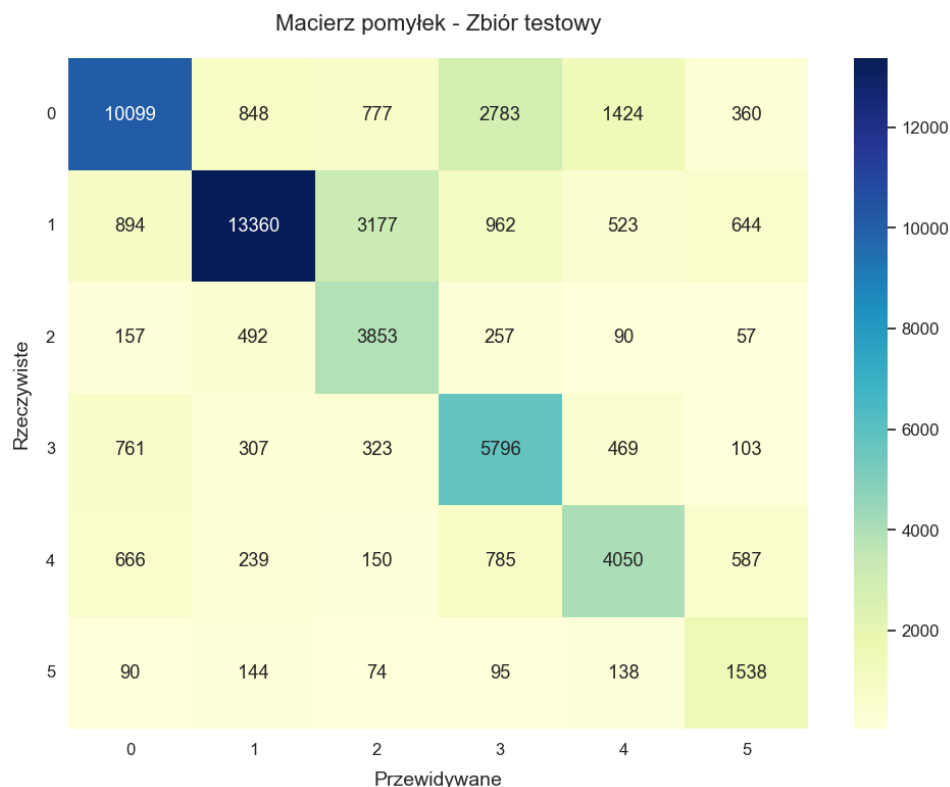
4.1.3. Podejścia osadzeń statycznych

Kolejny etap badań obejmował ewaluację modeli wykorzystujących statyczne osadzenia słów, zilustrowaną na przykładzie algorytmu Word2Vec. W przeciwieństwie do TF-IDF metoda ta pozwala na odwzorowanie relacji semantycznych między słowami poprzez umieszczenie ich w wielowymiarowej przestrzeni wektorowej. W ramach eksperymentu zbadano wpływ wymiarowości wektora na jakość klasyfikacji porównując osadzenia o rozmiarze 128 oraz 512 cech. Celem było sprawdzenie czy zwiększenie pojemności informacyjnej wektora przekłada się bezpośrednio na lepszą separowalność klas emocjonalnych. Zestawienie wyników uzyskane dla trzech badanych klasyfikatorów przedstawiono w Tabeli 4.3.

Tabela 4.3: Szczegółowe wyniki metryk na zbiorze testowym dla modelu Word2Vec (VECTOR_SIZE = 128)

Model	Accuracy	Balanced Accuracy	$F1_{macro}$
Logistic Regression	0.6578	0.6881	0.6396
XGBoost	0.5950	0.4098	0.4331
MLP	0.6780	0.7001	0.6433

Analiza metryk wskazuje, że dla wektora 128-wymiarowego najlepsze wyniki osiągnęła sieć neuronowa MLP, uzyskując wynik $F1_{macro} = 0.6433$. Warto również zwrócić uwagę na drastyczny spadek skuteczności algorytmu XGBoost, który przy tej reprezentacji danych uzyskał wyniki znacznie poniżej akceptowalnego poziomu, co sugeruje trudności w wyznaczaniu granic decyzyjnych na podstawie tak skonstruowanych cech. Ze względu na najlepsze rezultaty do dalszej analizy błędów wybrano model MLP, którego macierz pomyłek przedstawiono na Rysunku 4.4.



Rysunek 4.4: Macierz pomyłek dla osadzeń Word2Vec o wymiarze 128

Szczegółowa analiza macierzy pomyłek przedstawiona na Rysunku 4.4 wskazuje na istotne problemy z separacją niektórych klas. Model poprawnie klasyfikuje większość obserwacji należących do najliczniejszej klasy 1, lecz jednocześnie generuje dużą liczbę fałszywych przypisań do klasy 2. Zjawisko to jest jeszcze bardziej widoczne w przypadku klasy 0, dla której znaczna część próbek została błędnie zaklasyfikowana jako klasa 3. Wysoki poziom pomyłek pomiędzy konkretnymi parami klas sugeruje, że przy wymiarowości 128 wektory reprezentujące słowa kluczowe dla tych emocji znajdują się zbyt blisko siebie w przestrzeni wektorowej, co utrudnia poprawne działanie modelu. Uzyskane wyniki stanowią podstawę do weryfikacji, czy zwiększenie wymiarowości przestrzeni cech wpłynie na poprawę separacji problematycznych grup.

W kolejnym etapie analizy zwiększono wymiarowość przestrzeni wektorowej modelu Word2Vec do 512 cech. Celem było sprawdzenie czy większa pojemność wektora umożliwi zakodowanie bardziej złożonych relacji semantycznych, które mogły zostać pominięte w rzadszej przestrzeni. Wyniki dla tej konfiguracji zestawiono w Tabeli 4.4.

Tabela 4.4: Szczegółowe wyniki metryk na zbiorze testowym dla modelu Word2Vec(VECTOR_SIZE = 512)

Model	Accuracy	Balanced Accuracy	$F1_{macro}$
Logistic Regression	0.6881	0.7098	0.6459
XGBoost	0.6295	0.4616	0.4926
MLP	0.7375	0.7663	0.7041

Analiza wyników przedstawionych w Tabeli 4.4 wskazuje na poprawę skuteczności wszystkich modeli predykcyjnych. Zwiększenie wymiarowości przestrzeni cech umożliwiło uchwycenie bardziej złożonych zależności, co przełożyło się na lepszą jakość klasyfikacji. Najbardziej widoczny wzrost odnotowano w przypadku sieci neuronowej MLP, dla której wartość miary $F1_{macro}$ wzrosła do poziomu 0.7041, stanowiąc znaczącą poprawę względem wariantu 128-wymiarowego. Prostsze modele wykazały jedynie niewielką poprawę skuteczności, co wskazuje, że głównym ograniczeniem jest ich mniejsza zdolność do odwzorowania złożonych zależności, a nie jakość danych wejściowych. Na Rysunku 4.5 zaprezentowano macierz pomyłek dla najlepszego modelu MLP.



Rysunek 4.5: Macierz pomyłek dla osadzeń Word2Vec o wymiarze 512

Macierz pomyłek potwierdza, że zwiększenie wymiarowości przełożyło się na lepszą separowalność klas oraz wzrost liczby poprawnych predykcji dla większości emocji. Na szczególną uwagę zasługuje poprawa klasyfikacji klasy 0. Liczba poprawnych klasyfikacji wzrosła w tym przypadku do 11 804 (w porównaniu do 10 099 dla reprezentacji 128-wymiarowej), natomiast liczba błędnych przypisań do klasy 3 uległa wyraźnemu zmniejszeniu (z poziomu 2 783 do 1 312). Oznacza to, że 512-wymiarowy wektor skuteczniej wychwycił unikalne cechy semantyczne odróżniające te dwie emocje. Ostatecznie, pomimo ogólnej poprawy, model nadal wykazuje tendencję do mylenia klasy 1 z klasą 2, jednak skala tego zjawiska jest zdecydowanie mniejsza.

Zestawienie uzyskanych wyników prowadzi do wniosku, że w przypadku metody Word2Vec rozmiar wektora odgrywa istotną rolę w skuteczności modeli predykcyjnych. Zwiększenie wymiarowości z 128 do 512 oznacza czterokrotnie większą pojemność informacyjną wektora reprezentującego dane słowo. Dodatkowa przestrzeń umożliwiła zakodowanie znacznie większej liczby niuansów semantycznych, co w przypadku sieci MLP przełożyło się na wyraźny wzrost wartości wskaźnika $F1_{macro}$ z poziomu 0.6433 do 0.7041. Należy jednak podkreślić, że zwiększanie wymiarowości wektorów reprezentujących tekst nie stanowi gwarancji poprawy wyników, a o ostatecznej skuteczności decyduje również stopień dopasowania zastosowanej reprezentacji danych do architektury klasyfikatora. Kolejnym ograniczeniem jest fakt, że modele statyczne, takie jak Word2Vec, przypisują każdemu słowu jeden stały wektor, niezależnie od kontekstu. W konsekwencji, nawet wektory o dużej pojemności nie są w stanie w pełni oddać zmienności znaczeniowej danego słowa. W przypadku wyrażen wieloznacznych lub ironicznych model ten traci istotne informacje kontekstowe, co wyznacza górną granicę jego skuteczności.

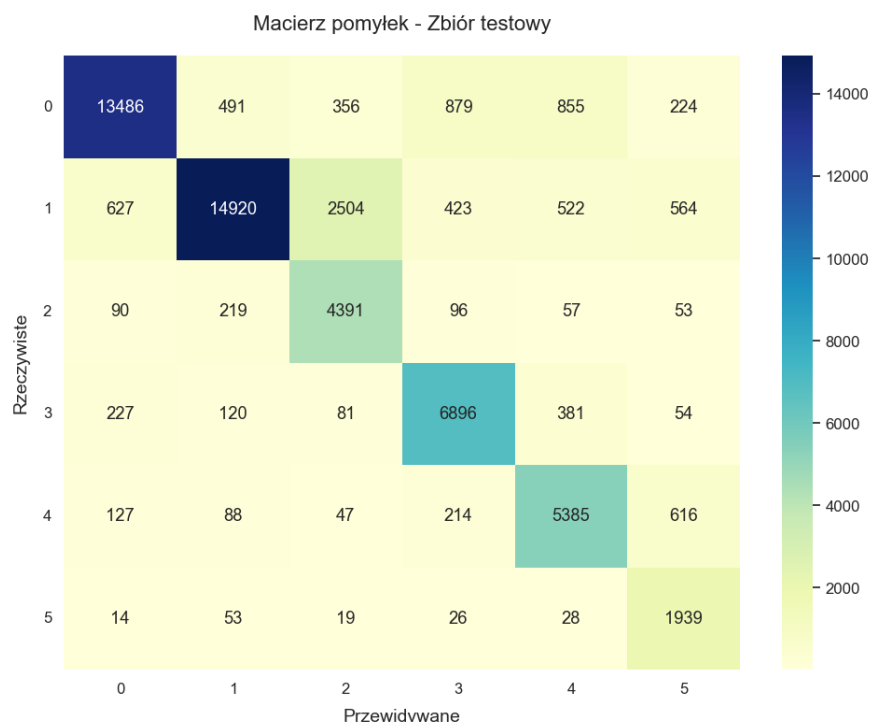
4.1.4. Podejścia osadzeń kontekstowych

Ostatni etap analizy obejmował weryfikację skuteczności modeli kontekstowych, które w odróżnieniu od wcześniej rozważanych podejść generują wektory w sposób dynamiczny, z uwzględnieniem całego zdania. Do porównania wybrano dwa modele osadzeń słów, w tym otwartoźródłowy model Sentence-BERT (SBERT), umożliwiający uruchomienie lokalne, oraz komercyjny model *text-embedding-3-small*, udostępniany za pośrednictwem interfejsu API firmy OpenAI. Celem tego porównania było sprawdzenie czy płatne i bardziej zaawansowane rozwiązania oferowane przez duże podmioty komercyjne zapewniają jakość niedostępną dla mniejszych, otwartoźródłowych architektur. Wyniki modelu OpenAI przedstawiono w Tabeli 4.5.

Tabela 4.5: Szczegółowe wyniki metryk dla modelu kontekstowego OpenAI na zbiorze testowym

Model	Accuracy	Balanced Accuracy	$F1_{macro}$
Logistic Regression	0.7744	0.7963	0.7407
XGBoost	0.7429	0.6461	0.6739
MLP	0.8238	0.8564	0.7939

Analiza wyników zaprezentowanych w Tabeli 4.5 jednoznacznie wskazuje na przewagę osadzeń kontekstowych nad wcześniej badanymi metodami. Najlepsze parametry klasyfikacji uzyskano ponownie przy użyciu sieci neuronowej MLP, osiągając wskaźnik $F1_{macro} = 0.7939$ na zbiorze testowym. Jest to wynik o blisko 9 punktów procentowych wyższy niż najlepszy rezultat uzyskany dla modelu Word2Vec, co świadczy o znacznie precyzyjniejszym odwzorowaniu znaczenia tekstu przez model kontekstowy. Warto również zauważyć, że nawet prostsze klasyfikatory, takie jak regresja logistyczna, osiągnęły wysokie wyniki predykcyjne, przewyższające najlepsze konfiguracje wcześniejszych podejść. Macierz pomyłek dla modelu OpenAI została przedstawiona na Rysunku 4.6.

**Rysunek 4.6:** Macierz pomyłek dla modelu kontekstowego OpenAI na zbiorze testowym

W przeciwieństwie do modeli statystycznych oraz metod opartych na statycznych osadzeniach słów, rozwiązanie wykorzystujące model OpenAI umożliwiło stosunkowo

skuteczną klasyfikację dwóch najsłabiej separowalnych klas, oznaczonych jako 0 i 1. Świadczy to o tym, że mechanizm uwagi stosowany w architekturze Transformer pozwala na poprawną interpretację głębokiego kontekstu wypowiedzi, a tym samym na rozróżnienie emocji, które na podstawie samych słów kluczowych mogą wydawać się zbliżone. Wysoka skuteczność oraz stabilność predykcji obserwowana jest we wszystkich klasach, co czyni ten model istotnym punktem odniesienia dla dalszych testów.

Drugim podejściem w grupie metod kontekstowych było wykorzystanie modelu Sentence BERT. Jest to rozwiązanie darmowe i ogólnodostępne pozwalające na generowanie osadzeń lokalnie bez konieczności komunikacji z zewnętrznym API OpenAI. Wyniki metryk dla tego modelu zestawiono w Tabeli 4.6.

Tabela 4.6: Szczegółowe wyniki metryk dla modelu kontekstowego Sentence BERT na zbiorze testowym

Model	Accuracy	Balanced Accuracy	$F1_{macro}$
Logistic Regression	0.6400	0.6537	0.5932
XGBoost	0.6550	0.5245	0.5523
MLP	0.7122	0.7318	0.6712

Analiza wyników zaprezentowanych w Tabeli 4.6 wskazuje, że model SBERT osiągnął niższą skuteczność predykcyjną niż rozwiązanie oferowane przez OpenAI. Najlepszy wynik metryki $F1_{macro}$ uzyskany przy użyciu sieci MLP wyniósł 0.6712. Jest to rezultat dobry, jednak zauważalnie niższy od wyniku osiągniętego przez model GPT (0.7939), a także nieznacznie słabszy od najlepszej konfiguracji statycznego modelu Word2Vec o wymiarze wektora 512 (0.7041). Należy jednak uwzględnić fakt, że SBERT jest rozwiązaniem całkowicie darmowym i możliwym do uruchomienia lokalnie. W tym kontekście osiągnięty wynik można uznać za satysfakcjonujący, stanowiący rozsądny kompromis dla systemów, w których priorytetem jest brak kosztów operacyjnych przy zachowaniu przyzwoitej jakości klasyfikacji. W celu zweryfikowania natury błędów klasyfikacji wygenerowano macierz pomyłek, przedstawioną na Rysunku 4.7.



Rysunek 4.7: Macierz pomyłek dla modelu kontekstowego Sentence-BERT na zbiorze testowym

Macierz pomyłek wskazuje, że model SBERT charakteryzuje się niższą skutecznością w zakresie separacji trudnych klas w porównaniu z jego większym, komercyjnym odpowiednikiem. Ponownie widoczny jest problem błędnego klasyfikowania klasy 0 jako klasy 1, co skutkuje ponad 2000 nieprawidłowych przypisań. Charakter rozkładu błędów jest zbliżony do obserwowanego w metodach statycznych, co potwierdza, że w przypadku mniejszych modeli językowych sama architektura Transformer nie gwarantuje automatycznie skutecznego rozróżniania niuansów semantycznych, jeśli model bazowy nie dysponuje wystarczająco bogatą reprezentacją danych dziedzinowych.

Zestawienie wyników modeli kontekstowych prowadzi do sformułowania kluczowych wniosków końcowych. Biorąc pod uwagę wyłącznie jakość klasyfikacji, najwyższą skuteczność wykazał model OpenAI. Jako jedyny spośród wszystkich analizowanych rozwiązań osiągnął wyniki zbliżone do poziomu 0.80 dla metryki $F1_{macro}$, przewyższając nie tylko model SBERT, ale również wcześniejsze metody statyczne (Word2Vec) oraz statystyczne (TF-IDF).

Model OpenAI, dzięki bardzo dużej bazie wiedzy, na której został wytrenowany, skutecznie rozwiązał problem rozróżniania zbliżonych do siebie emocji, z czym inne rozwiązania miały duże trudności. Należy jednak ocenić te wyniki w szerszym kontekście, uwzględniając koszty oraz środowisko wdrożeniowe. Model Sentence BERT, mimo słabszych wyników, posiada szereg unikalnych zalet w systemach produkcyjnych. Na szczególną uwagę zasługuje fakt, że struktura jego błędów jest bardzo podobna do

modelu OpenAI, ponieważ model myli te same klasy, jednak z nieznacznie większą częstotliwością. Jednocześnie jest to rozwiązanie całkowicie darmowe, które nie wymaga przesyłania wrażliwych danych użytkownika do zewnętrznej infrastruktury obliczeniowej. Zapewnia to pełną prywatność oraz bezpieczeństwo danych i zgodność z coraz częściej spotykanymi rygorystycznymi politykami bezpieczeństwa. Dodatkowo generowanie wektorów odbywa się lokalnie, co eliminuje opóźnienia sieciowe i umożliwia działanie aplikacji bez dostępu do internetu.

Podsumowując, wybór pomiędzy analizowanymi technologiami stanowi klasyczny kompromis. W zastosowaniach krytycznych, w których priorytetem jest możliwie najwyższa precyzja rozumienia tekstu, uzasadnione jest poniesienie kosztów związanych z wykorzystaniem modelu OpenAI. Natomiast w typowych scenariuszach aplikacji użytkowych, gdzie kluczowe znaczenie mają szybkość działania, możliwość pracy w trybie bez dostępu do sieci oraz brak stałych kosztów operacyjnych, model SBERT pozostaje funkcjonalną i bezpieczną alternatywą, oferującą akceptowalny poziom skuteczności. Mając na uwadze powyższe czynniki, model Sentence BERT został wybrany do implementacji w finalnej aplikacji użytkowej.

Po wyborze modelu przeprowadzono szczegółową analizę przypadków uznanych za poważne błędy, polegających na przypisaniu obserwacji do innej kategorii emocji niż etykieta referencyjna. Analiza ta ujawniła istotne niespójności w samym zbiorze danych, a niekoniecznie w procesie wnioskowania modelu. Tabela 4.7 zawiera przykłady, w których predykcje sieci neuronowej wydają się lepiej odzwierciedlać semantyczne znaczenie tekstu niż przypisane im oryginalne etykiety.

Tabela 4.7: Przykłady błędnej klasyfikacji dla par emocji o niezblizonej semantyce

text	true_label	pred_label	confidence
i feel like crap by fab o lens	Joy	Sadness	0.9876
i was having a pretty low day this week and feeling pretty rubbish about everything dont you just hate that	Joy	Sadness	0.9844
i have a good feeling the raiders will end up being the read more on a rel nofollow target blank href http sports	Sadness	Joy	0.9574
i still have a lot of time ahead of me but as time goes by i start to feel more jaded towards things	Sadness	Joy	0.9545

W pierwszej grupie przykładów znajdują się teksty oznaczone w zbiorze uczącym jako klasa „Joy”, które zawierają wyraźne sformułowania o negatywnym sentymencie. Przykładem jest zdanie zawierające frazę „*i feel like crap*”, które model zaklasyfikował jako „Sadness” z bardzo wysoką pewnością, przekraczającą 98%. W takich przypadkach model prawidłowo zidentyfikował negatywny sentyment wypowiedzi, co świadczy o poprawnej zdolności generalizacji, natomiast błąd klasyfikacji wynika z występowania w zbiorze danych błędów anotacyjnych. Analogiczną sytuację obserwuje się w kierunku przeciwnym, w którym teksty oznaczone w zbiorze danych jako „Sadness” zostały przez model zaklasyfikowane jako „Joy”. Prowadzi to do wniosku, że prezentowane w raporcie metryki ewaluacyjne mogą być w rzeczywistości zaniżone na skutek błędów ludzkich popełnionych na etapie tworzenia zbioru danych. W wielu przypadkach model wykazuje większą trafność w interpretacji treści tekstu, niż wynikałoby to z bezpośredniego porównania z etykietami referencyjnymi, co potwierdza skuteczność zastosowanej architektury opartej na osadzeniach słów SBERT oraz wykorzystanych modeli predykcyjnych w poprawnym rozpoznawaniu intencji emocjonalnych w tekście.

Druga kategoria analizowanych pomyłek obejmuje błędy o mniejszej wadze, występujące pomiędzy klasami o zgodnym sentymencie. Przedstawione w Tabeli 4.8 zestawienie dotyczy pomyłek pomiędzy klasami „Love” oraz „Joy”, które są ze sobą ściśle powiązane znaczeniowo.

Tabela 4.8: Przykłady błędnej klasyfikacji dla par emocji o zbliżonej semantyce

text	true_label	pred_label	confidence
i wear it i feel lovely and classy and grown up	Love	Joy	0.9808
i work hard and when my films do well i feel accepted	Love	Joy	0.9785
i feel that since then we have been much more considerate and loving toward each other	Joy	Love	0.9896
i have the day off tomorrow however and im feeling a craving for that delicious vindaloo	Joy	Love	0.9850

W przeciwieństwie do błędnej klasyfikacji emocji pozytywnych jako negatywnych, w tym przypadku model poprawnie rozpoznaje charakter wypowiedzi, a rozbieżność dotyczy jedynie specyfiki dominującej emocji. Główną przyczyną tego typu rozbieżności jest wysokie podobieństwo semantyczne analizowanych stanów emocjonal-

nych oraz częściowe nakładanie się ich znaczeń. Model uczony na dużych korpusach tekstowych wykazuje tendencję do traktowania klasy „Love” jako specyficznego podzbioru szerszej kategorii „Joy”. Widoczne w Tabeli 4.8 przykłady, w których tekst oznaczony etykietą „Love” został sklasyfikowany jako „Joy”, często zawierają słownictwo typowe dla ogólnego poczucia szczęścia i satysfakcji, co utrudnia precyzyjne rozróżnienie tych klas bez uwzględnienia szerszego kontekstu sytuacyjnego.

Tabela 4.9 przedstawia wyniki wszystkich wykorzystanych w analizie metod reprezentacji tekstu wraz z zastosowanymi modelami klasyfikacyjnymi, zarówno na zbiorze treningowym, walidacyjnym, jak i testowym.

Tabela 4.9: Zbiorcze zestawienie wyników wszystkich modeli przed korektą

Reprezentacja	Zbiór	Regresja Logistyczna			XGBoost			Sieci MLP		
		Acc	Bal. Acc	F1	Acc	Bal. Acc	F1	Acc	Bal. Acc	F1
TF-IDF	Train	0.6749	0.7005	0.6588	0.6383	0.5917	0.6285	0.6769	0.7117	0.6607
	Valid	0.6632	0.6878	0.6466	0.6322	0.5836	0.6191	0.6604	0.6922	0.6432
	Test	0.6628	0.6855	0.6452	0.6293	0.5790	0.6151	0.6578	0.6881	0.6396
Word2Vec (Rozmiar wektora = 128)	Train	0.5570	0.5734	0.5086	0.6271	0.4464	0.4783	0.7132	0.7552	0.6811
	Valid	0.5575	0.5741	0.5092	0.5928	0.4052	0.4267	0.6857	0.7097	0.6450
	Test	0.5548	0.5703	0.5064	0.5950	0.4098	0.4331	0.6840	0.7067	0.6422
Word2Vec (Rozmiar wektora = 512)	Train	0.6310	0.6544	0.5864	0.6736	0.5191	0.5606	0.7586	0.7974	0.7300
	Valid	0.6307	0.6544	0.5860	0.6298	0.4629	0.4943	0.7409	0.7708	0.7080
	Test	0.6254	0.6475	0.5798	0.6295	0.4616	0.4926	0.7375	0.7663	0.7041
BERT	Train	0.6041	0.6297	0.5612	0.6574	0.5063	0.5450	0.7272	0.7916	0.7034
	Valid	0.5982	0.6206	0.5541	0.6164	0.4505	0.4759	0.6857	0.7270	0.6522
	Test	0.5991	0.6232	0.5558	0.6159	0.4528	0.4799	0.6876	0.7278	0.6534
RoBERTa	Train	0.5810	0.6053	0.5379	0.6333	0.4746	0.5075	0.6756	0.7177	0.6443
	Valid	0.5762	0.5997	0.5333	0.6089	0.4443	0.4711	0.6586	0.6906	0.6234
	Test	0.5775	0.6003	0.5343	0.6122	0.4486	0.4757	0.6600	0.6933	0.6245
Sentence BERT	Train	0.6402	0.6706	0.6008	0.7068	0.5815	0.6246	0.7385	0.7753	0.7063
	Valid	0.6395	0.6693	0.5995	0.6729	0.5356	0.5732	0.7116	0.7303	0.6698
	Test	0.6360	0.6668	0.5963	0.6736	0.5372	0.5744	0.7122	0.7318	0.6712
GPT	Train	0.7813	0.8076	0.7501	0.7560	0.6599	0.6965	0.8612	0.8981	0.8360
	Valid	0.7770	0.7996	0.7437	0.7185	0.6052	0.6398	0.8308	0.8582	0.8004
	Test	0.7744	0.7963	0.7407	0.7209	0.6101	0.6441	0.8298	0.8557	0.7986

Najlepsze rezultaty w zadaniu predykcyjnym uzyskano dla konfiguracji wykorzystującej reprezentację wektorową generowaną przez model OpenAI w połączeniu z siecią neuronową, która osiągnęła na zbiorze testowym zbalansowaną dokładność na poziomie 85,57%. Drugie pod względem skuteczności okazało się rozwiązanie oparte na modelu Sentence-BERT, uzyskujące wynik 73,18%. Wyniki te potwierdzają, że zaawansowane metody kontekstowe, uwzględniające głęboki kontekst semantyczny, znacząco przewyższają klasyczne podejścia w zadaniu klasyfikacji emocji.

Ze względu na zidentyfikowany problem błędnych etykiet w zbiorze danych przeprowadzono procedurę automatycznej weryfikacji i korekty oznaczeń. Proces ten oparto na analizie spójności semantycznej z wykorzystaniem algorytmu k najbliższych

sąsiadów (ang. *k-nearest neighbors*) w przestrzeni wektorowej, przyjmując parametr $k = 20$ oraz metrykę kosinusową.

Zdefiniowano regułę konfliktu, zgodnie z którą, jeżeli dla danej obserwacji znaleziono przynajmniej jednego sąsiada o podobieństwie ≥ 0.74 posiadającego inną etykietę, wszystkie takie rekordy kwalifikowano do ponownej oceny. Wyselekcjonowane w ten sposób zdania poddano korekcie oznaczeń przy użyciu modelu GPT-4o-mini, wykorzystując jego zdolności generalizacji do rozstrzygania niejednoznaczności. W wyniku tej operacji spośród 41 022 analizowanych rekordów zmieniono etykietę w 7127 przypadkach, co pozwoliło na wyeliminowanie błędów i zwiększenie jakości danych treningowych przed etapem uczenia właściwego modelu.

Porównanie wyników uzyskanych przed oraz po przeprowadzeniu procedury korekty etykietowania danych wykazuje jednoznaczny, pozytywny wpływ tej operacji na zdolności generalizacyjne badanych modeli. Dzięki wyeliminowaniu błędów w ponad 7 000 rekordów większość analizowanych konfiguracji odnotowała istotny wzrost kluczowych metryk jakości. Szczegółowe zestawienie uzyskanych wyników zaprezentowano w Tabeli 4.10.

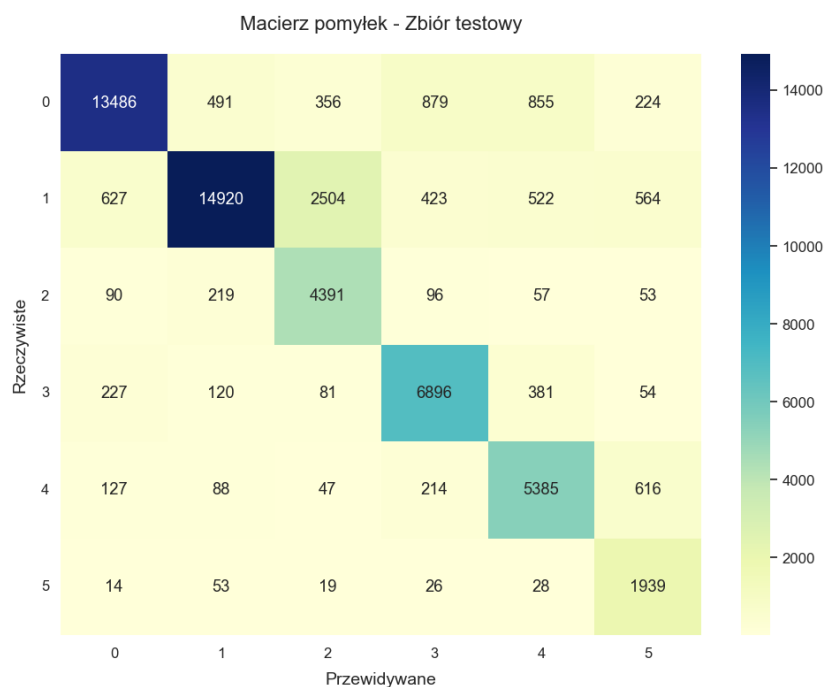
Tabela 4.10: Zbiorcze zestawienie wyników wszystkich modeli po korekcie

Reprezentacja	Zbiór	Regresja Logistyczna			XGBoost			Sieci MLP		
		Acc	Bal. Acc	F1	Acc	Bal. Acc	F1	Acc	Bal. Acc	F1
TF-IDF	Train	0.7055	0.7278	0.6742	0.6703	0.6154	0.6753	0.7148	0.7357	0.6849
	Valid	0.6845	0.7030	0.6506	0.6565	0.6050	0.6564	0.6891	0.7049	0.6557
	Test	0.7024	0.7177	0.6694	0.6587	0.6356	0.6627	0.7047	0.7153	0.6719
Word2Vec (Rozmiar wektora = 128)	Train	0.6042	0.6277	0.5463	0.6643	0.4903	0.5362	0.6753	0.6908	0.6388
	Valid	0.5967	0.6161	0.5410	0.6142	0.4299	0.4616	0.6575	0.6662	0.6195
	Test	0.6069	0.6339	0.5491	0.6267	0.4396	0.4742	0.6739	0.6874	0.6352
Word2Vec (Rozmiar wektora = 512)	Train	0.6988	0.7264	0.6479	0.6972	0.5406	0.5901	0.7632	0.8004	0.7423
	Valid	0.6887	0.7126	0.6396	0.6437	0.4739	0.5122	0.7567	0.7832	0.7145
	Test	0.7035	0.7314	0.6521	0.6566	0.4813	0.5212	0.7432	0.7732	0.7192
BERT	Train	0.6002	0.6264	0.5577	0.6253	0.4578	0.4894	0.7329	0.7937	0.7081
	Valid	0.5931	0.6163	0.5497	0.6039	0.4304	0.4541	0.6855	0.7137	0.6556
	Test	0.5944	0.6187	0.5512	0.6024	0.4302	0.4546	0.6850	0.7149	0.6657
RoBERTa	Train	0.6048	0.6362	0.5525	0.6562	0.4950	0.5364	0.6963	0.7235	0.6723
	Valid	0.5945	0.6213	0.5448	0.6231	0.4537	0.4869	0.6766	0.7106	0.6541
	Test	0.6068	0.6390	0.5536	0.6398	0.4694	0.5050	0.6797	0.7113	0.6589
Sentence BERT	Train	0.6632	0.6830	0.6085	0.6874	0.5424	0.5831	0.7451	0.7826	0.7075
	Valid	0.6521	0.6638	0.5970	0.6614	0.5070	0.5416	0.7149	0.7295	0.6670
	Test	0.6662	0.6873	0.6112	0.6752	0.5210	0.5566	0.7339	0.7558	0.6879
GPT	Train	0.7990	0.8285	0.7641	0.7983	0.7207	0.7590	0.8626	0.8985	0.8327
	Valid	0.7838	0.8066	0.7470	0.7456	0.6397	0.6790	0.8268	0.8512	0.7897
	Test	0.8023	0.8286	0.7667	0.7651	0.6677	0.7062	0.8470	0.8706	0.8124

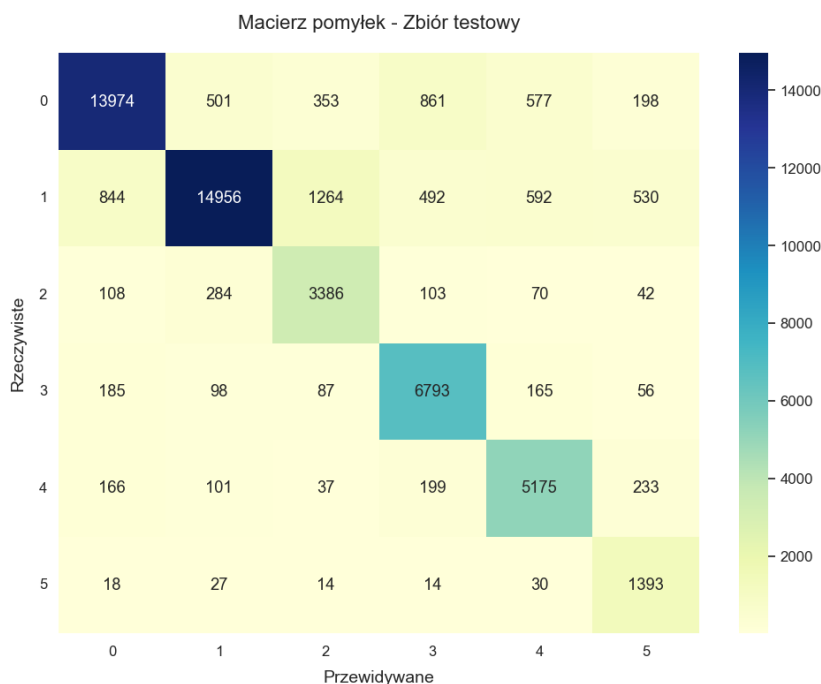
Najlepsze rozwiązanie, oparte na reprezentacji GPT oraz sieci neuronowej MLP, poprawiło wartość metryki $F1_{macro}$ na zbiorze testowym z 79,86% do 81,24%. Wyraźny przyrost skuteczności zaobserwowano również w przypadku prostszych klasyfi-

katorów, które są bardziej wrażliwe na obecność szumu w danych. Uzyskane rezultaty potwierdzają skuteczność zastosowanej procedury weryfikacji danych opartej na algorytmie KNN oraz korekcie oznaczeń z wykorzystaniem modelu GPT-4o-mini.

Rysunek 4.8 przedstawia macierz pomyłek wygenerowaną dla modelu OpenAI na zbiorze testowym, przy wykorzystaniu oryginalnych danych. Macierz pomyłek otrzymana po przeprowadzeniu procesu korekty etykiet (Rysunek 4.9) wskazuje na wyraźną poprawę skuteczności tego modelu, szczególnie w obszarach generujących wcześniej największą liczbę błędów. Liczba przypadków, w których klasa 1 była błędnie klasyfikowana jako klasa 2, zmniejszyła się niemal o połowę, z 2 504 do 1 264. Jednocześnie poprawie uległo rozpoznawanie pozostałych kategorii. Przykładowo dla klasy 0 liczba poprawnych klasyfikacji wzrosła do 13 974. Zmiany w łącznej liczbie obserwacji w poszczególnych wierszach macierzy wynikają z procesu, w ramach którego część obserwacji zmieniła przypisaną kategorię lub została usunięta.



Rysunek 4.8: Macierz pomyłek dla modelu kontekstowego OpenAI na zbiorze testowym



Rysunek 4.9: Macierz pomyłek dla modelu kontekstowego OpenAI po poprawieniu etykiet na zbiorze testowym

Porównanie macierzy pomyłek przed i po przeprowadzeniu korekty etykiet potwierdza zasadność zastosowanego procesu poprawy etykietowania danych. Usunięcie błędnych oznaczeń w zbiorze treningowym ograniczyło wpływ niepoprawnych wzorców na proces uczenia, co przełożyło się na bardziej jednoznaczne rozróżnianie klas. Jednocześnie korekta etykiet w zbiorze testowym zwiększyła rzetelność oceny dopasowania modelu, dzięki czemu raportowane wartości metryk ewaluacyjnych w większym stopniu odzwierciedlają rzeczywistą zdolność modelu do generalizacji, a nie jego dopasowanie do zaszumionych danych.

4.2. Weryfikacja jakości modeli tłumaczeniowych

Kolejnym kluczowym etapem prac była implementacja modułu odpowiedzialnego za tłumaczenie maszynowe. Krok ten był niezbędny, ponieważ główny model klasyfikacyjny został wytrenowany na danych w języku angielskim. Implementację modułu tłumaczącego poprzedzono weryfikacją jakości trzech różnych architektur, których celem była obsługa użytkowników polskojęzycznych. Zastosowanie wielojęzycznych modeli tłumaczenia maszynowego umożliwiła wprowadzanie tekstu w innych językach, co zwiększa uniwersalność aplikacji. W obszarze przetwarzania języka naturalnego algorytmy analizy tekstu najczęściej osiągają najwyższą skuteczność dla języka angielskiego.

go, co wynika z dominacji anglojęzycznych korpusów treningowych. W konsekwencji języki słabiej reprezentowane lub charakteryzujące się odmienną strukturą gramatyczną są gorzej obsługiwane przez modele ogólne, a zastosowanie translacji jako etapu wstępnego staje się warunkiem uzyskania satysfakcjonujących rezultatów klasyfikacji.

W tej części pracy do treningu modułu tłumaczącego wykorzystano zbiór danych zawierający równoległe pary zdań w języku polskim i angielskim, pochodzący z zasobów ManyThings.org, będących częścią projektu Tatoeba [14]. Zbiór ten obejmuje około 51 000 par zdań, które stanowią wysokiej jakości materiał treningowy dla systemu tłumaczenia maszynowego z języka polskiego na język angielski. Dane mają charakter tekstowy i zostały zapisane w formacie TSV, umożliwiającym łatwe przetwarzanie w środowisku Python oraz integrację z bibliotekami do uczenia maszynowego.

Zbiór został wybrany ze względu na szeroki zakres tematyczny, wiarygodność oraz dostępność na licencji Creative Commons (CC-BY), umożliwiającej legalne wykorzystanie danych do celów badawczych i komercyjnych. Dane te odzwierciedlają rzeczywiste użycie języka w codziennej komunikacji oraz jego zróżnicowanie składniowe i semantyczne, co sprzyja budowie modeli dobrze uogólniających w zadaniach tłumaczenia maszynowego. Każdy rekord w zbiorze danych odpowiada pojedynczej parze zdań, w której pierwsze zdanie zapisane jest w języku polskim, a drugie w języku angielskim, przy czym są one oddzielone znakiem tabulatora. Taka struktura umożliwia bezpośrednie wykorzystanie zbioru do trenowania, walidacji oraz testowania modeli tłumaczenia maszynowego. Dane posłużyły do opracowania, wytrenowania oraz oceny skuteczności modelu tłumaczenia opartego na architekturze Transformera. Wszystkie etapy przetwarzania danych, ich segmentacji oraz trenowania modelu zostały zrealizowane w środowisku Visual Studio Code z wykorzystaniem języka Python.

Do oceny jakości generowanych tłumaczeń wykorzystano metrykę BLEU (ang. *Bilingual Evaluation Understudy*), która stanowi obecnie standard w dziedzinie tłumaczenia maszynowego. Jest to miara automatyczna, która określa stopień podobieństwa między tekstem wygenerowanym przez model, a jednym lub kilkoma tłumaczeniami wzorcowymi. Wartość metryki BLEU zawiera się w przedziale $[0, 1]$ (lub $0 - 100\%$), gdzie wyższa wartość oznacza większą zgodność z tłumaczeniem referencyjnym [24]. Algorytm opiera się na obliczaniu precyzji n -gramów, co pozwala na ocenę zarówno doboru słownictwa, jak i poprawności frazeologicznej. Ostateczny wynik jest średnią geometryczną zmodyfikowanych precyzji n -gramów, skorygowaną o tzw. karę za krótkość (ang. *Brevity Penalty*, BP). Wzór na metrykę BLEU definiuje się następująco

$$BLEU = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right),$$

gdzie p_n oznacza zmodyfikowaną precyzję obliczaną dla n -gramów o długości n , natomiast w_n to waga przypisana tym n -gramom, która standardowo przyjmuje wartość $1/N$. Parametr N reprezentuje maksymalną długość analizowanego ciągu słów. Całość wyrażenia jest modyfikowana przez współczynnik BP , czyli karę za krótkość. Współczynnik ten zapobiega faworyzowaniu przez model nienaturalnie krótkich tłumaczeń, które mogłyby sztucznie zawyżać precyzję, promując tym samym wyniki o długości adekwatnej do oryginału. Tabela 4.11 przedstawia porównanie wartości metryki BLEU dla analizowanych modeli na zbiorze testowym.

Tabela 4.11: Wyniki metryk dla modeli tłumaczenia maszynowego na zbiorze testowym

Model	BLEU
Własna architektura	10.68
M2M100	39.76
mBART	59.46

Na podstawie danych zaprezentowanych w Tabeli 4.11 można zaobserwować, że najniższy wynik (10.68 BLEU) uzyskała autorska architektura trenowana od podstaw, co stanowi punkt odniesienia dla modeli pre-trenowanych. Znaczącą poprawę jakości odnotowano w przypadku modelu M2M100 (39.76 BLEU), jednak zdecydowanie najlepsze rezultaty osiągnął model mBART, uzyskując wynik na poziomie 59.46 BLEU. Ze względu na najwyższą zdolność do poprawnej translacji, to właśnie model mBART został wybrany jako architektura bazowa do realizacji aplikacji. W celu dokładnego zrozumienia charakterystyki analizowanego zbioru danych w Tabeli 4.12 przedstawiono analizę jakościową wygenerowanych predykcji w modelu mBART.

Tabela 4.12: Analiza jakościowa wygenerowanych predykcji

Źródło (PL)	Model (Predykcja)	Referencja (EN)
Nikt nie pływa w lagunie, ponieważ jest brudna.	Nobody swims in the lagoon because it's dirty.	Nobody swims in the lagoon because it's filthy.
Skoro nie chcesz tego robić, to w porządku.	If you don't want to do that, that's fine.	If you don't want to do it, it's OK.
Byłem trochę głodny.	I was a little hungry.	I was a bit hungry.
Skaleczyłam się.	I was rocking.	I've injured myself.
Naprawdę nie chcę tego robić.	I really don't want to do that.	I really don't want to do this.

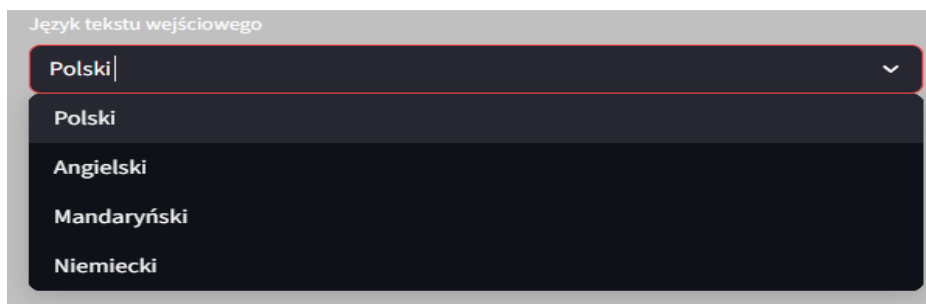
Analiza wyników przedstawionych w Tabeli 4.12 wskazuje, że model mBART charakteryzuje się wysoką zdolnością rozumienia języka angielskiego. Model nie dokonuje mechanicznego tłumaczenia, ale jest w stanie dobrać trafne synonimy, na przykład zamieniając „filthy” na lepsze „dirty” czy „it’s OK” na naturalne „that’s fine”, zachowując przy tym poprawną gramatykę języka. Z drugiej strony wiersz w Tabeli 4.12, wyróżniony kolorem czerwonym, ilustruje istotne ograniczenie analizowanego modelu. W przypadku krótkiego zdania „Skaleczyłam się” model nie dokonał poprawnej translacji, lecz wygenerował zdanie „I was rocking”, które mimo poprawności gramatycznej nie odpowiada znaczeniu oryginału. Obserwowane zjawisko wskazuje na tendencję modelu do generowania treści opartej na domyślnych wzorcach językowych w sytuacjach, w których dostępny kontekst semantyczny jest niewystarczający.

4.3. Implementacja modułu rozpoznawania emocji i tłumaczenia tekstu

Integralną częścią pracy jest implementacja opracowanego modelu klasyfikacyjnego oraz modelu tłumaczenia maszynowego w formie interaktywnej aplikacji. Narzędzie to zostało zbudowane w języku Python z wykorzystaniem biblioteki Streamlit, co umożliwiło sprawne stworzenie estetycznego interfejsu graficznego dedykowanego rozwiązaniom uczenia maszynowego. Jako środowisko hostingu oraz platformę zapewniającą niezbędną moc obliczeniową wybrano usługę Hugging Face Spaces, która zapewnia bezpośrednią integrację z repozytorium modeli sztucznej inteligencji. Interfejs użytkownika umożliwia wybór języka źródłowego oraz wprowadzenie tekstu wejściowego, który następnie poddawany jest automatycznej walidacji pod kątem minimalnej długości. Wypowiedzi zawierające mniej niż siedem słów nie są przekazywane do dalszej analizy i wymagają rozszerzenia przez użytkownika. Po uruchomieniu analizy system realizuje w tle wieloetapowy proces przetwarzania danych.

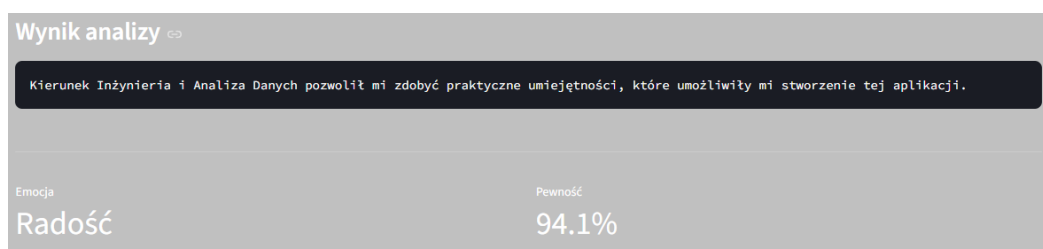
W pierwszym etapie tekst wejściowy jest tłumaczony na język angielski z wykorzystaniem modelu mBART-50, co stanowi niezbędny etap normalizacji dla dalszego przetwarzania. Następnie przetłumaczona treść przekazywana jest do modelu SBERT, gdzie tworzona jest jej wektorowa reprezentacja semantyczna. Ostatecznie uzyskane wektory trafiają do sieci neuronowej MLP, która generuje końcową predykcję. Wynikiem działania aplikacji jest wyświetlenie nazwy dominującej emocji wraz z wartością procentową określającą pewność predykcji modelu. Dodatkowo użytkownik otrzymuje pełny podgląd wyników w formie wykresu słupkowego, który obrazuje rozkład prawdopodobieństwa wystąpienia każdej z sześciu zdefiniowanych klas emocjonalnych dla badanego tekstu. Rysunek 4.10 przedstawia element interfejsu graficznego, który umożli-

liwia użytkownikowi wybranie języka tekstu wprowadzanego do analizy emocji. System obsługuje cztery języki naturalne: polski, angielski, mandaryński oraz niemiecki.



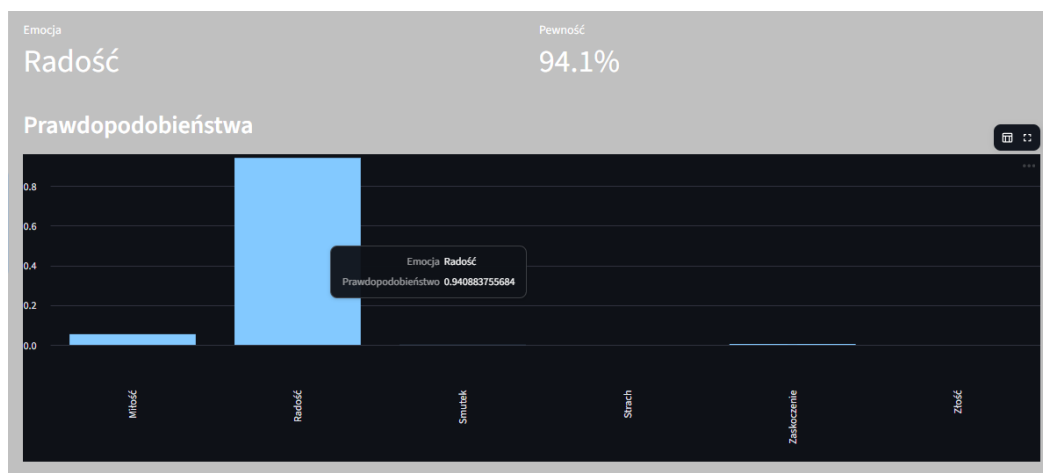
Rysunek 4.10: Wybór języka źródłowego tekstu poddawanego analizie emocji przez użytkownika.

Rysunek 4.11 przedstawia kluczowy etap interakcji z systemem. Po zdefiniowaniu języka źródłowego użytkownik wprowadza analizowany tekst do dedykowanego pola, przy czym aplikacja weryfikuje spełnienie warunku minimalnej długości wypowiedzi. Proces analizy tekstu inicjowany jest poprzez naciśnięcie przycisku „Analiza emocji występujących w tekście”. W odpowiedzi system generuje sekcję wynikową, prezentującą zidentyfikowaną klasę emocjonalną z największym prawdopodobieństwem (w wskazanym przykładzie jest to Radość) oraz pewność predykcji wyrażoną w procentach.



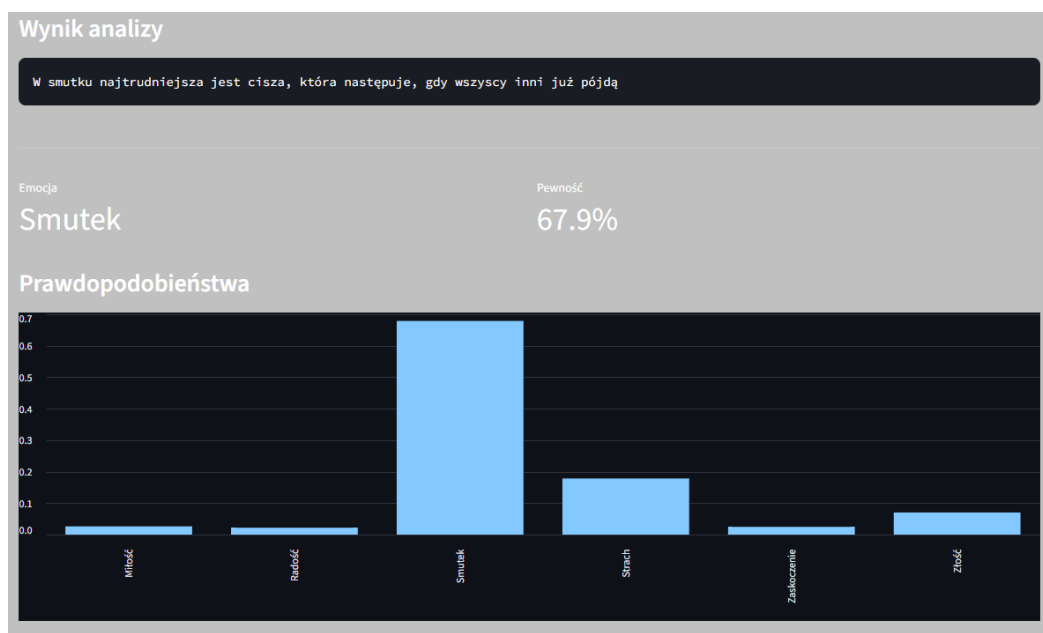
Rysunek 4.11: Prezentacja działania aplikacji dla przykładowego tekstu w języku polskim

Integralną częścią procesu interpretacji wyników jest graficzna reprezentacja otrzymanych rezultatów, przedstawiona na Rysunku 4.12. Wygenerowany wykres słupkowy obrazuje szczegółowy rozkład prawdopodobieństwa przypisany przez model do każdej z sześciu zdefiniowanych kategorii (Miłość, Radość, Smutek, Strach, Zaskoczenie, Złość). Zaprezentowana wizualizacja umożliwia ocenę stopnia jednoznaczności predykcji poprzez analizę rozkładu prawdopodobieństw przypisanych poszczególnym kategoriom emocji. Interaktywny charakter wykresu wspiera szczegółową interpretację wyników, umożliwiając wyświetlenie dokładnych wartości prawdopodobieństwa po najechaniu kursorem na wybrany słupek. W analizowanym przypadku dla klasy Radość wartość ta wynosi około 0.94.



Rysunek 4.12: Wizualizacja pełnego rozkładu prawdopodobieństwa dla poszczególnych klas emocji

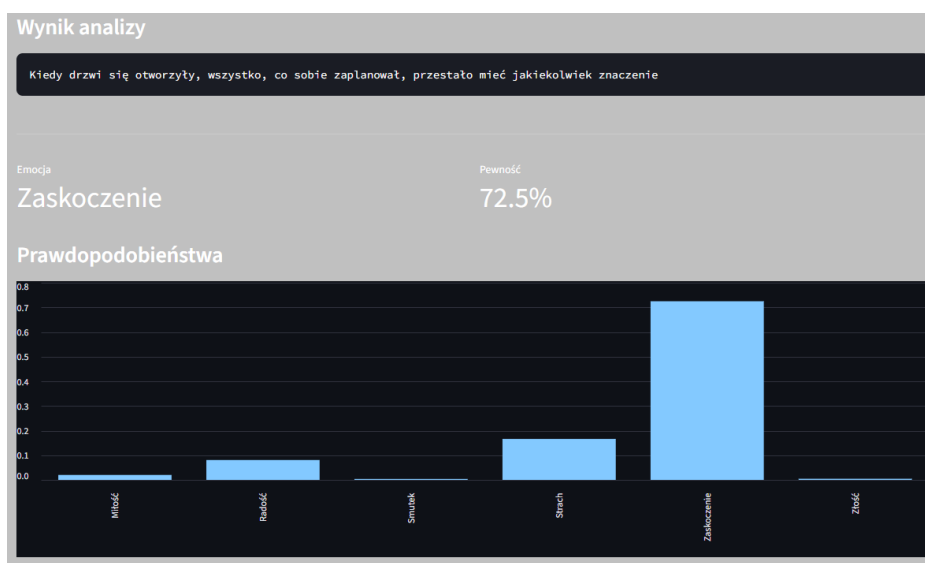
Na Rysunku 4.13 przedstawiono przykładowy wynik działania aplikacji dla zdania o negatywnym charakterze emocjonalnym. Dla wprowadzonego tekstu wejściowego model sklasyfikował dominującą emocję jako Smutek, przypisując jej prawdopodobieństwo na poziomie 67,9%. Wizualizacja w formie wykresu słupkowego pokazuje również, że drugą najbardziej prawdopodobną klasą jest Strach, podczas gdy udział emocji pozytywnych (Radość, Miłość) pozostaje marginalny.



Rysunek 4.13: Zrzut ekranu aplikacji przedstawiający wynik klasyfikacji emocji Smutek

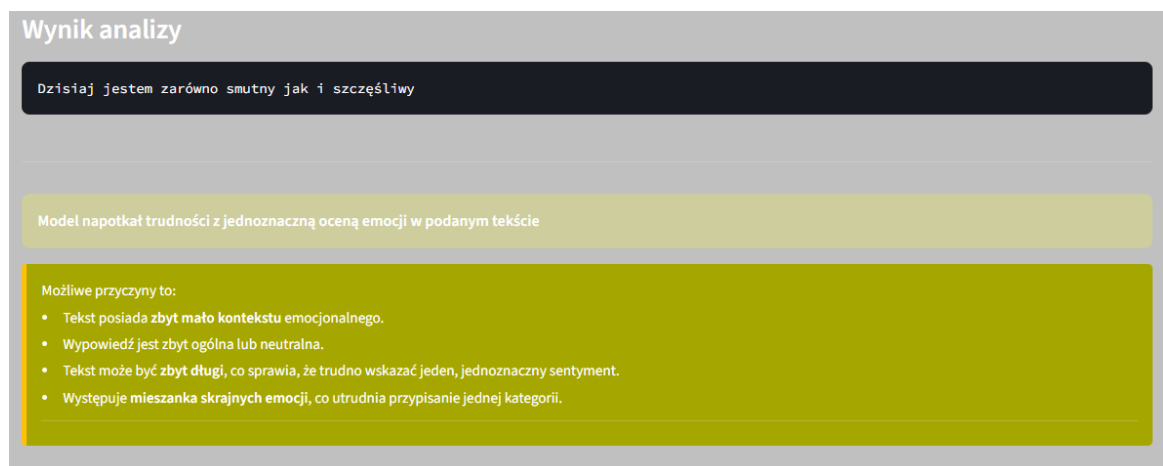
Rysunek 4.14 ilustruje działanie modelu w przypadku tekstu o dynamicznej zmianie kontekstu. Dla zdania wejściowego opisującego nagły zwrot akcji system przypisał

najwyższe prawdopodobieństwo klasie Zaskoczenie, uzyskując wartość na poziomie 72,5%. Warto również zwrócić uwagę na drugi co do wielkości słupek odpowiadający emocji Strach, którego udział wynosi około 17%. Taki rozkład jest semantycznie uzasadniony, ponieważ sytuacja, w której dotychczasowe plany tracą znaczenie w momencie otwarcia drzwi, wiąże się z elementem niepewności i napięcia. Zjawisko to zostało przez model zinterpretowane jako współwystępowanie emocji zaskoczenia i lęku. Pozostałe emocje, w tym Radość, uzyskały marginalne wartości prawdopodobieństwa, co potwierdza wysoką precyzję klasyfikatora.



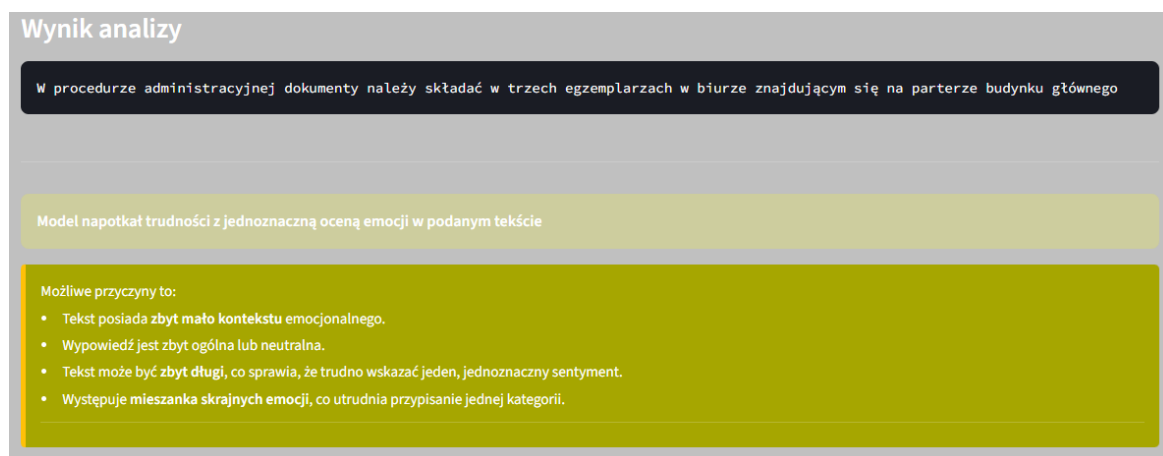
Rysunek 4.14: Zrzut ekranu aplikacji prezentujący wynik klasyfikacji dla emocji Zaskoczenie

Rysunek 4.15 przedstawia działanie aplikacji w sytuacji, gdy wprowadzony tekst zawiera wzajemnie wykluczające się emocje. W takim przypadku model nie dokonuje wymuszonej klasyfikacji o niskim poziomie pewności, lecz uruchamia mechanizm obsługi niejednoznaczności. Użytkownik otrzymuje informację zwrotną wskazującą, że jednoczesne występowanie skrajnych emocji uniemożliwia jednoznaczne przypisanie poprawnego wyniku.



Rysunek 4.15: Zrzut ekranu aplikacji prezentujący wynik klasyfikacji dla kombinacji emocji Smutek oraz Radość jednocześnie

Na Rysunku 4.16 przedstawiono wynik analizy dla zdania o charakterze urzędowym. Tego typu wypowiedzi cechują się stylem bezosobowym oraz brakiem słów i konstrukcji wyrażających wyraźne emocje. Model prawidłowo nie wykrył dominującej emocji, co skutkowało wyświetleniem komunikatu o braku jednoznacznej klasyfikacji. System trafnie wskazał, że analizowana wypowiedź jest zbyt ogólna lub neutralna. Takie zachowanie potwierdza zdolność algorytmu do rozróżniania treści emocjonalnych i neutralnych, co zapobiega błędnemu przypisywaniu emocji w przypadkach ich faktycznego braku.



Rysunek 4.16: Zrzut ekranu aplikacji prezentujący wynik klasyfikacji dla tekstu bez emocji

Podsumowując, opracowana aplikacja stanowi praktyczny rezultat realizacji pracy, łącząc zaawansowane algorytmy klasyfikacji emocji z modułem tłumaczenia maszynowego oraz przejrzystym interfejsem użytkownika. Przeprowadzone testy wykazały,

że system działa zgodnie z przyjętymi założeniami oraz poprawnie rozpoznaje kluczowe emocje występujące w analizowanych wypowiedziach. Istotną zaletą aplikacji jest jej zdolność do identyfikowania sytuacji neutralnych, a także tekstów zawierających wzajemnie wykluczające się emocje. W sytuacjach niepewności system informuje użytkownika o braku rozstrzygnięcia, co zwiększa wiarygodność narzędzia i zapobiega wprowadzaniu użytkownika w błąd. Dodatkowo udostępnienie aplikacji w chmurze Hugging Face zapewnia łatwy dostęp do systemu oraz stwarza możliwość jego dalszego rozwoju w przyszłości.

Podsumowanie i wnioski

Praca dotyczy analizy skuteczności metod osadzania słów w zadaniach przetwarzania języka naturalnego. W części teoretycznej zdefiniowano dziedzinę przetwarzania języka naturalnego, omówiono jej rolę oraz ewolucję, a także wskazano różnice w sposobie interpretacji tekstu przez człowieka oraz systemy komputerowe. Ponadto szczegółowo omówiono trzy główne metody reprezentacji tekstu, obejmujące podejścia statystyczne (BoW, TF-IDF, n -gramy), statyczne modele wektorowe (Word2Vec, GloVe, FastText) oraz kontekstowe metody reprezentacji oparte na architekturze Transformer. W ramach ostatniej grupy opisano zasady działania mechanizmów uwagi oraz scharakteryzowano modele takie jak ELMo, BERT, RoBERTa i GPT.

Kolejnym zagadnieniem omówionym w części teoretycznej były wybrane architektury tłumaczenia maszynowego oraz algorytmy uczenia maszynowego stosowane w klasyfikacji tekstu. W ramach architektur tłumaczenia maszynowego przedstawiono założenia oraz mechanizmy działania nowoczesnych modeli, takich jak mBART oraz M2M-100. W przypadku tradycyjnych algorytmów scharakteryzowano trzy główne grupy modeli: modele liniowe, drzewiaste oraz sieci neuronowe, wraz z ich zaletami oraz potencjalnymi ograniczeniami.

W części praktycznej przeprowadzono badania empiryczne polegające na klasyfikacji tekstu ze względu na zawarte w nim emocje. Głównym celem badań było zweryfikowanie skuteczności wybranych metod reprezentacji tekstu w połączeniu z różnymi algorytmami uczenia maszynowego, w tym regresją logistyczną, XGBoost oraz sieciami neuronowymi, przy zachowaniu spójnych architektur i konfiguracji parametrów. Wyniki modeli porównano z wykorzystaniem specjalnie dobranych metryk (F1-macro, Accuracy oraz Balanced Accuracy) na dwóch zbiorach danych. Pierwszym z nich był zbiór oryginalny, natomiast drugi stanowił zbiór, w którym zastosowano automatyczną korektę błędnych oznaczeń danych przy użyciu samodzielnie opracowanego systemu opartego na modelu GPT-4o-mini.

Rezultaty przeprowadzonych analiz pozwoliły stwierdzić, że kontekstowe reprezentacje tekstu, w szczególności oparte na architekturze Transformer, w porównaniu

z metodami statystycznymi oraz statycznymi, charakteryzują się wyższą skutecznością niezależnie od zastosowanego modelu klasyfikacyjnego. Dodatkowo zaobserwowano, że w większości przypadków sieci neuronowe umożliwiały uzyskanie najlepszych wyników dzięki swojej zdolności do modelowania złożonych, nieliniowych wzorców. Wykorzystanie skorygowanego zbioru danych pozwoliło ponadto na uzyskanie istotnie lepszych wyników predykcyjnych w porównaniu do początkowego zbioru. W tej części pracy porównano również skuteczność wybranych architektur przeznaczonych do tłumaczenia maszynowego, spośród których najlepsze wyniki uzyskał model mBART.

Podsumowując, finalnym wynikiem pracy była implementacja wielojęzycznej aplikacji w środowisku Streamlit, która integruje automatyczne tłumaczenie tekstu oraz wykorzystuje model do klasyfikacji emocji. Przeprowadzone testy potwierdziły, że wytrenowane modele mogą być skutecznie wykorzystywane w praktycznych systemach analitycznych, umożliwiając predykcję emocji w tekście niezależnie od języka wprowadzenia danych. Aplikacja została wdrożona na platformie Hugging Face, co zapewnia łatwy dostęp do systemu oraz stwarza możliwości jego dalszego rozwoju i rozszerzania funkcjonalności w przyszłości.

Bibliografia

- [1] M. Awad, R. Khanna, *Efficient Learning Machines*, Apress, Berkeley, 2015
- [2] D. Bahdanau, K. Cho, Y. Bengio, *Neural Machine Translation by Jointly Learning to Align and Translate*, Proceedings of the 3rd International Conference on Learning Representations, San Diego, 2015
- [3] P. Bojanowski, E. Grave, A. Joulin, T. Mikolov, *Enriching Word Vectors with Subword Information*, Transactions of the Association for Computational Linguistics, Stroudsburg, 2017, 135–146
- [4] P. F. Brown, S. A. Della Pietra, V. J. Della Pietra, R. L. Mercer, *The Mathematics of Statistical Machine Translation: Parameter Estimation*, Computational Linguistics, Cambridge, 1993, 263–311
- [5] T. Chen, C. Guestrin, *XGBoost: A Scalable Tree Boosting System*, Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, 2016, 785–794
- [6] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Minneapolis, 2019, 4171–4186
- [7] A. Elkomy, *Twitter Emotion Dataset*, Kaggle, <https://www.kaggle.com/datasets/adhamelkomy/twitter-emotion-dataset>
- [8] A. Fan et al., *Beyond English-Centric Multilingual Machine Translation*, Journal of Machine Learning Research, Brookline, 2021
- [9] J. H. Friedman, *Greedy Function Approximation: A Gradient Boosting Machine*, The Annals of Statistics, Beachwood, 2001, 1189–1232
- [10] M. Grandini, E. Bagli, G. Visani, *Metrics for Multi-Class Classification: an Overview*, arXiv preprint, 2020
- [11] J. Hutchins, *The History of Machine Translation in a Nutshell*, Self-published, Norwich, 2005
- [12] A. Joulin, E. Grave, P. Bojanowski, T. Mikolov, *Bag of Tricks for Efficient Text Classification*, Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics, Walencja, 2017, 427–431

- [13] D. Jurafsky, J. H. Martin, *Speech and Language Processing*, Stanford University, Stanford, 2023
- [14] C. Kelly, *Tab-delimited Bilingual Sentence Pairs (pol-eng) – ManyThings (Tatoeba)*, ManyThings.org, Toyota, 2025
- [15] D. Khurana, A. Koli, K. Khatte, S. Singh, *Natural Language Processing: State of The Art, Current Trends and Challenges*, Multimedia Tools and Applications, New York, 2023, 3713–3744
- [16] N. Kriegeskorte, T. Golan, *Neural network models and deep learning: A primer for biologists*, Current Biology, Cambridge, 2019, 231–236
- [17] Y. Liu et al., *RoBERTa: A Robustly Optimized BERT Pretraining Approach*, Open Access Library Journal, 2019
- [18] Y. Liu et al., *Multilingual Denoising Pre-training for Neural Machine Translation*, Transactions of the Association for Computational Linguistics, Stroudsburg, 2020, 726–742
- [19] W.Y. Loh, *Classification and Regression Trees*, Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery, Hoboken, 2011, 14–23
- [20] C. D. Manning, H. Schütze, *Foundations of Statistical Natural Language Processing*, MIT Press, Cambridge, 1999
- [21] T. Mikolov, K. Chen, G. S. Corrado, J. Dean, *Efficient Estimation of Word Representations in Vector Space*, Proceedings of the International Conference on Learning Representations, Scottsdale, 2013, 1–12
- [22] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, J. Dean, *Distributed Representations of Words and Phrases and their Compositionality*, Advances in Neural Information Processing Systems, Lake Tahoe, 2013, 3111–3119
- [23] G. Neubig, *Neural Machine Translation and Sequence-to-sequence Models: A Tutorial*, Computing Research Repository, 2017
- [24] K. Papineni, S. Roukos, T. Ward, W.J. Zhu, *BLEU: a method for automatic evaluation of machine translation. Proceedings of the 40th Annual Meeting on Association for Computational Linguistics*, Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, Philadelphia, 2002, 311–318
- [25] J. Pennington, R. Socher, C. D. Manning, *GloVe: Global Vectors for Word Representation*, Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, Doha, 2014, 1532–1543
- [26] M. E. Peters et al., *Deep Contextualized Word Representations*, Association for Computational Linguistics, New Orleans, Louisiana, 2018
- [27] J. R. Quinlan, *Induction of Decision Trees*, Machine Learning, Boston, 1986, 81–106
- [28] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever, *Improving Language Understanding by Generative Pre-Training*, OpenAI Technical Report, San Francisco, 2018

- [29] K. Sarıtaş, C. A. Öz, T. Güngör, *A Comprehensive Analysis of Static Word Embeddings for NLP*, Engineering Applications of Artificial Intelligence, Amsterdam, 2024
- [30] I. Sutskever, O. Vinyals, Q. V. Le, *Sequence to Sequence Learning with Neural Networks*, Advances in Neural Information Processing Systems, Montreal, 2014, 3104–3112
- [31] W. Van Woensel, S. Motie, *A Systematic Review on Process Extraction using Natural Language Processing with Rule-based, Machine and Deep Learning Methods*, Enterprise Information Systems, Abingdon, 2024
- [32] A. Vaswani et al., *Attention Is All You Need*, Advances in Neural Information Processing Systems, Long Beach, 2017, 5998–6008
- [33] Z. Xu, M. Chen, K. Q. Weinberger, F. Sha, *An alternative text representation to TF-IDF and Bag-of-Words*, arXiv preprint, Los Angeles, 2013
- [34] A. Zhang, Z. C. Lipton, M. Li, A. J. Smola, *Dive into Deep Learning*, Cambridge University Press, Cambridge, 2023
- [35] X. Zhang, E. Milios, *Exploration of TF-IDF and alternatives*, arXiv preprint, 2023
- [36] K. Zheng et al., *ExLM: Rethinking the Impact of [MASK] Tokens in Masked Language Models*, Proceedings of the 42nd International Conference on Machine Learning, Vancouver, 2025

Spis rysunków

1.1	Ewolucja NLP - główne etapy rozwoju	9
1.2	Porównanie: klasyczne programowanie vs uczenie maszynowe	11
2.1	Przykład relacji wektorowych w przestrzeni statycznych osadzeń słów	19
3.1	Schemat przykładowego drzewa decyzyjnego	36
3.2	Schemat sieci MLP z jedną warstwą ukrytą	37
4.1	Rozkład liczby obserwacji w poszczególnych klasach emocji	40
4.2	Schemat interpretacji macierzy pomyłek dla problemu klasyfikacji 6-klasowej . .	44
4.3	Macierz pomyłek dla modelu TF-IDF na zbiorze testowym	46
4.4	Macierz pomyłek dla osadzeń Word2Vec o wymiarze 128	48
4.5	Macierz pomyłek dla osadzeń Word2Vec o wymiarze 512	49
4.6	Macierz pomyłek dla modelu kontekstowego OpenAI na zbiorze testowym	51
4.7	Macierz pomyłek dla modelu kontekstowego Sentence-BERT na zbiorze testowym	53
4.8	Macierz pomyłek dla modelu kontekstowego OpenAI na zbiorze testowym	58
4.9	Macierz pomyłek dla modelu kontekstowego OpenAI po poprawieniu etykiet na zbiorze testowym	59
4.10	Wybór języka źródłowego tekstu poddawanego analizie emocji przez użytkownika.	63
4.11	Prezentacja działania aplikacji dla przykładowego tekstu w języku polskim . . .	63
4.12	Wizualizacja pełnego rozkładu prawdopodobieństwa dla poszczególnych klas emocji	64
4.13	Zrzut ekranu aplikacji przedstawiający wynik klasyfikacji emocji Smutek	64
4.14	Zrzut ekranu aplikacji prezentujący wynik klasyfikacji dla emocji Zaskoczenie . .	65
4.15	Zrzut ekranu aplikacji prezentujący wynik klasyfikacji dla kombinacji emocji Smutek oraz Radość jednocześnie	66
4.16	Zrzut ekranu aplikacji prezentujący wynik klasyfikacji dla tekstu bez emocji . .	66

Spis tabel

4.1	Przykłady identycznych tekstów przypisanych do różnych klas	41
4.2	Szczegółowe wyniki metryk dla modeli opartych na TF-IDF na zbiorze testowym	45
4.3	Szczegółowe wyniki metryk na zbiorze testowym dla modelu Word2Vec (VECTOR_SIZE = 128)	47
4.4	Szczegółowe wyniki metryk na zbiorze testowym dla modelu Word2Vec(VECTOR_SIZE = 512)	49
4.5	Szczegółowe wyniki metryk dla modelu kontekstowego OpenAI na zbiorze testowym	51
4.6	Szczegółowe wyniki metryk dla modelu kontekstowego Sentence BERT na zbiorze testowym	52
4.7	Przykłady błędnej klasyfikacji dla par emocji o niezbliżonej semantyce	54
4.8	Przykłady błędnej klasyfikacji dla par emocji o zbliżonej semantyce	55
4.9	Zbiorcze zestawienie wyników wszystkich modeli przed korektą	56
4.10	Zbiorcze zestawienie wyników wszystkich modeli po korekcie	57
4.11	Wyniki metryk dla modeli tłumaczenia maszynowego na zbiorze testowym . . .	61
4.12	Analiza jakościowa wygenerowanych predykcji	61

Załączniki

1. Płyta CD zawiera niniejszą pracę w wersji elektronicznej oraz kod źródłowy przygotowany w środowisku Visual Studio Code obejmujący wyniki wszystkich obliczeń i przeprowadzonych testów wykorzystanych w pracy.

Streszczenie (Summary)

Analiza skuteczności wybranych metod osadzania słów w zadaniach przetwarzania języka naturalnego

W pracy zaprezentowano analizę skuteczności wybranych metod osadzania słów w zadaniach przetwarzania języka naturalnego. Szczegółowo omówiono metody reprezentacji tekstu, obejmujące podejścia statystyczne, statyczne modele wektorowe oraz modele kontekstowe oparte na architekturze Transformer. Przedstawiono również architektury wykorzystywane w tłumaczeniu maszynowym oraz algorytmy uczenia maszynowego stosowane w klasyfikacji tekstu. W części praktycznej przeprowadzono zadanie detekcji emocji na podstawie tekstu oraz porównano skuteczność różnych modeli predykcyjnych i reprezentacji wektorowych. Istotną część pracy stanowiła weryfikacja jakości danych, obejmująca identyfikację oraz automatyczną korektę błędnych oznaczeń, realizowaną z wykorzystaniem samodzielnie opracowanego systemu opartego na modelu GPT-4o-mini. Wynikiem pracy było opracowanie wielojęzycznej aplikacji służącej do wykrywania emocji na podstawie tekstu.

Analysis of the Effectiveness of Selected Word Embedding Methods in Natural Language Processing Tasks

The study presents an analysis of the effectiveness of selected word embedding methods in natural language processing tasks. Methods of text representation are discussed in detail, including statistical approaches, static vector models, and contextual models based on the Transformer architecture. Architectures used in machine translation as well as machine learning algorithms applied to text classification are also presented. In the practical part, an emotion detection task based on text was conducted, and the effectiveness of various predictive models and vector representations was compared. An important part of the study involved data quality verification, including the identification and automatic correction of erroneous labels, carried out using

a self-developed system based on the GPT-4o-mini model. The outcome of the study was the development of a multilingual application for emotion detection based on text.